Table of Contents

Contributors

User	👍, Answers
Arjun Suresh	17841, 221
Praveen Saini	4669, 61
Gate Keeda	2717, 58
Akash Kanase	2145, 35
Deepak Poonia	1739, 28
Digvijay	1649, 27
Rajesh Pradhan	1597, 22
Vikrant Singh	1515, 17
Amar Vashishth	1481, 20
gatecse	1480, 20
Prashant Singh	1474, 28
Sachin Mittal	1463, 18
dd	1248, 10
Bhagirathi Nayak	1239, 21
Rajarshi Sarkar	948, 22
Kalpna Bhargav	924, 11
Shaik Masthan	919, 24
Sankaranarayanan P.N	890, 19
Pooja Palod	850, 15
Ahwan Mishra	786, 9
Rishabh Gupta	767, 3
Hira	670, 28
Ankit Rokde	667, 10
Pragy Agarwal	641, 6
Manu Thakur	626, 10
Anurag Semwal	592, 7
Abhilash Panicker	528, 5
Mithlesh Upadhyay	520, 7
Monanshi Jain	506, 8
minal	490, 7
jayendra	472, 9
Akhil Nadh PC	459, 6
Anoop Sonkar	425, 6
ryan sequeira	414, 4
Sandeep_Uniyal	405, 7
Shubham Srivastava	394, 5
suraj	374, 6
Srinath Jayachandran	370, 2
Arnabi Bej	367, 4
Ashish Deshmukh	366, 2
Aditi Dan	364, 4

User	?Added
Kathleen Bankson	444
Arjun Suresh	267
GO Editor	179
Ishrat Jahan	123
gatecse	83
Misbah	77
Rucha Shelke	33
admin	27
Akash Kanase	27
Sandeep Singh	26
Madhav kumar	11
khush tak	8

User	✎Done
Lakshman Patel (GATE And Tech)	458
kenzou	450
Milicevic3306	202
Arjun Suresh	199
gatecse	115
Hira	108
soujanyareddy13	79
Naveen Kumar	76
Pooja Khatri	72
V S Silpa	58
Shubham Sharma	53
Shaik Masthan	48
Misbah	48
Shikha Mallick	46
Shri 1	40
Deepak Poonia	31
shadymademe	26
Akash Dinkar	25
Umesh Shelke	20
GO Editor	18
Krithiga2101	18
jothee_new	16
Ankit Singh	15
Sweta Kumari Rawani	14
Abhrajyoti Kundu	11
Sachin Mittal	9
Kavali Yedidyah Sagar	9
Sujith K	9
srestha	8
Mk Utkarsh	8
Sukanya Das	6
Ajay kumar soni	6
chinmay jain	6
goku4199	5
Puja Mishra	5

1 Algorithms (358)

Searching, Sorting, Hashing, Asymptotic worst case time and Space complexity, Algorithm design techniques: Greedy, Dynamic programming, and Divide-and-conquer, Graph search, Minimum spanning trees, Shortest paths.

Mark Distribution in Previous GATE

Year	2026 - 1	2026 - 2	2025 - 1	2025 - 2	2024 - 1	2024 - 2	2023	2022	2021 - 1	2021 - 2	Minimum
1 Mark Count	4	4	2	2	1	2	2	2	3	2	1
2 Marks Count	6	4	3	3	4	2	2	2	3	4	2
Total Marks	16	12	8	8	9	6	6	6	9	10	**6**

Welcome to the "Algorithms" chapter, a cornerstone of Computer Science and a high-scoring section in the GATE examination. This subject delves into the systematic procedures and computational methods used to solve problems, focusing on their efficiency and correctness. Mastering algorithms is crucial not only for theoretical understanding but also for practical application in software development and system design. In GATE CS, algorithms typically carry a significant weightage, often ranging from 10 to 15 marks, with questions spanning theoretical concepts, time and space complexity analysis, problem-solving, and application of various design techniques. Expect a mix of Multiple Choice Questions (MCQ), Multiple Select Questions (MSQ), and Numerical Answer Type (NAT) questions, requiring a deep understanding of underlying principles and the ability to analyze and compare different algorithmic approaches.

Topic-wise Key Concepts

Algorithm Design

Definition: Algorithm design is the process of creating a well-defined computational procedure to solve a specific problem. It involves understanding the problem, choosing appropriate data structures, and devising a step-by-step method that is correct, efficient, and terminates.

- **Properties/Identities:**
  - **Correctness:** An algorithm must produce the correct output for all valid inputs.
  - **Efficiency:** Measured by time and space complexity.
  - **Finiteness:** Must terminate after a finite number of steps.
  - **Definiteness:** Each step must be precisely defined.
  - **Input/Output:** Must take zero or more inputs and produce one or more outputs.
- **Pitfalls/Tricks:**
  - Overlooking edge cases or constraints that might break the algorithm's logic.
  - Designing an algorithm that is correct but too inefficient for practical use or given constraints.
- **Techniques/Shortcuts:**
  - Start with a brute-force solution, then optimize.
  - Break down complex problems into smaller, manageable subproblems.
  - Consider different data structures and their impact on performance.

Algorithm Design Techniques

Definition: These are general strategies or paradigms used to develop algorithms for a wide range of problems. Key techniques include Divide and Conquer, Greedy Algorithms, Dynamic Programming, Backtracking, and Branch and Bound.

- **Properties/Identities:**
  - **Divide and Conquer:** Divide problem into subproblems, solve recursively, combine solutions.
  - **Greedy Algorithms:** Make locally optimal choices at each step, hoping to find a global optimum.
  - **Dynamic Programming:** Solves problems with overlapping subproblems and optimal substructure by storing results of subproblems.
- **Pitfalls/Tricks:**
  - Mistaking a problem for one solvable by a greedy approach when it requires dynamic programming.
  - Incorrectly identifying optimal substructure or overlapping subproblems.
- **Techniques/Shortcuts:**
  - For DP, identify the state, recurrence relation, and base cases.
  - For Greedy, prove the greedy choice property and optimal substructure.

Asymptotic Notations

Definition: Mathematical notations used to describe the limiting behavior of a function when the argument tends towards a particular value or infinity, primarily used to classify algorithms by their running time or space requirements as input size grows.

- **Formulas/Theorems:**
  - **Big-O Notation (Upper Bound):** $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.
  - **Omega Notation (Lower Bound):** $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$.
  - **Theta Notation (Tight Bound):** $f(n) = \Theta(g(n))$ if there exist positive constants c_1, c_2 and n_0 such that $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$.
  - **Little-o Notation (Strict Upper Bound):** $f(n) = o(g(n))$ if $\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$.
  - **Little-omega Notation (Strict Lower Bound):** $f(n) = \omega(g(n))$ if $\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$.
- **Properties/Identities:**
  - Reflexivity: $f(n) = O(f(n))$, $f(n) = \Omega(f(n))$, $f(n) = \Theta(f(n))$.
  - Transitivity: If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$. (Applies to all notations).
  - Symmetry: $f(n) = \Theta(g(n))$ iff $g(n) = \Theta(f(n))$.
  - Transpose Symmetry: $f(n) = O(g(n))$ iff $g(n) = \Omega(f(n))$. Similarly for o and ω.
  - Sum Rule: If $f(n) = O(g(n))$ and $h(n) = O(g(n))$, then $f(n) + h(n) = O(g(n))$. (Take the maximum order term).
  - Product Rule: If $f(n) = O(g(n))$ and $h(n) = O(p(n))$, then $f(n) \cdot h(n) = O(g(n) \cdot p(n))$.
- **Pitfalls/Tricks:**
  - Confusing Big-O with Theta: Big-O is an upper bound, Theta is a tight bound.
  - Ignoring constants and lower-order terms: Asymptotic notation focuses on growth rate.
  - Assuming $O(f(n))$ means exactly $f(n)$ operations; it means "at most proportional to $f(n)$".
- **Techniques/Shortcuts:**
  - For polynomials, the highest degree term dominates. E.g., $3n^2 + 5n + 10 = O(n^2)$.
  - Logarithms grow slower than any polynomial: $\log n = O(n^\epsilon)$ for any $\epsilon > 0$.
  - Exponentials grow faster than any polynomial: $n^k = O(a^n)$ for any $a > 1$.

Bellman Ford

Definition: A single-source shortest path algorithm that can handle graphs with negative edge weights. It works by repeatedly relaxing all edges in the graph, detecting negative cycles if they exist.

- **Formulas/Theorems:**
  - **Relaxation Step:** For an edge (u, v) with weight $w(u, v)$, if $d[u] + w(u, v) < d[v]$, then $d[v] = d[u] + w(u, v)$.
  - **Number of Iterations:** $|V| - 1$ iterations are sufficient to find shortest paths in a graph with $|V|$ vertices, assuming no negative cycles reachable from the source.
  - **Negative Cycle Detection:** After $|V| - 1$ iterations, if any edge can still be relaxed in the $|V|$-th iteration, a negative cycle exists.
- **Properties/Identities:**
  - Works on directed and undirected graphs (undirected edges treated as two directed edges).
  - Can detect negative cycles.
  - Time Complexity: $O(|V| \cdot |E|)$.
  - Space Complexity: $O(|V|)$ for distance array, $O(|V|)$ for predecessor array.
- **Pitfalls/Tricks:**
  - Incorrectly assuming it's faster than Dijkstra's for non-negative weights (Dijkstra's is $O(E \log V)$ or $O(E + V \log V)$).
  - Forgetting to initialize distances (source to 0, others to infinity).
- **Techniques/Shortcuts:**
  - Visualize the relaxation process step-by-step.
  - For negative cycle detection, run one extra iteration after $|V| - 1$ iterations.

Binary Heap

Definition: A complete binary tree that satisfies the heap property: for a min-heap, every node's value is less than or equal to its children's values; for a max-heap, every node's value is greater than or equal to its children's values.

Typically implemented using an array.

- **Formulas/Theorems:**
  - **Parent of node i:** $\lfloor (i-1)/2 \rfloor$ (for 0-indexed array).
  - **Left child of node i:** $2i+1$ (for 0-indexed array).
  - **Right child of node i:** $2i+2$ (for 0-indexed array).
  - **Height of a heap with N nodes:** $\lfloor \log_2 N \rfloor$.
- **Properties/Identities:**
  - **Min-Heap:** Root is the minimum element.
  - **Max-Heap:** Root is the maximum element.
  - **Complete Binary Tree:** All levels are completely filled except possibly the last level, which is filled from left to right.
  - **Operations Time Complexity:**
    - Insert: $O(\log N)$
    - Delete-Min/Max: $O(\log N)$
    - Build-Heap (from an array of N elements): $O(N)$
- **Pitfalls/Tricks:**
  - Confusing min-heap with max-heap properties.
  - Incorrectly calculating parent/child indices, especially for 1-indexed vs. 0-indexed arrays.
- **Techniques/Shortcuts:**
  - When building a heap, start from the last non-leaf node and call `heapify` upwards.
  - Heap sort uses a max-heap.

Binary Search

Definition: An efficient algorithm for finding an item from a sorted list of items. It repeatedly divides the search interval in half, eliminating half of the remaining elements at each step.

- **Formulas/Theorems:**
  - **Recurrence Relation:** $T(N) = T(N/2) + O(1)$.
  - **Time Complexity:** $O(\log N)$.
  - **Space Complexity:** $O(1)$ (iterative) or $O(\log N)$ (recursive due to call stack).
- **Properties/Identities:**
  - Requires the input array/list to be sorted.
  - Can be used to find the first/last occurrence of an element, or the ceiling/floor.
- **Pitfalls/Tricks:**
  - Forgetting to handle edge cases like empty arrays or target not found.
  - Off-by-one errors in index calculations (e.g., `mid = (low + high) / 2`). Use `mid = low + (high - low) / 2` to prevent overflow.
- **Techniques/Shortcuts:**
  - Always ensure the search space is correctly updated (`low = mid + 1` or `high = mid - 1`).
  - For finding first/last occurrence, adjust the update logic to keep searching in the relevant half even after finding a match.

Binary Search Tree (BST)

Definition: A binary tree where for each node, all values in its left subtree are less than the node's value, and all values in its right subtree are greater than the node's value. No duplicate values are typically allowed.

- **Properties/Identities:**
  - **Inorder Traversal:** Produces elements in sorted order.
  - **Average Time Complexity:**
    - Search, Insert, Delete: $O(\log N)$
  - **Worst-Case Time Complexity (skewed tree):**
    - Search, Insert, Delete: $O(N)$
  - **Space Complexity:** $O(N)$ for storing nodes.
- **Pitfalls/Tricks:**
  - Forgetting to handle the case of deleting a node with two children (replace with inorder successor/predecessor).
  - Assuming $O(\log N)$ performance always; a skewed BST degrades to $O(N)$.
- **Techniques/Shortcuts:**
  - To find minimum, traverse left until null. To find maximum, traverse right until null.

- For deletion, the inorder successor is the smallest node in the right subtree.

Binary Tree

Definition: A tree data structure where each node has at most two children, referred to as the left child and the right child. It's a fundamental non-linear data structure.

- **Formulas/Theorems:**
  - **Maximum nodes at level i (root at level 0):** 2^i.
  - **Maximum nodes in a binary tree of height h:** $2^{h+1} - 1$.
  - **Minimum height of a binary tree with N nodes:** $\lceil \log_2(N+1) \rceil - 1$.
- **Properties/Identities:**
  - **Full Binary Tree:** Every node has either 0 or 2 children.
  - **Complete Binary Tree:** All levels are completely filled except possibly the last level, which is filled from left to right.
  - **Perfect Binary Tree:** All internal nodes have two children and all leaves are at the same level. (A perfect binary tree is always full and complete).
  - **Skewed Binary Tree:** All nodes have only one child (either left or right).
- **Pitfalls/Tricks:**
  - Confusing different types of binary trees (full, complete, perfect).
  - Incorrectly calculating height or number of nodes for specific tree types.
- **Techniques/Shortcuts:**
  - Understand the relationship between height and number of nodes for different tree types.
  - Practice drawing trees from traversals.

Breadth First Search (BFS)

Definition: A graph traversal algorithm that explores all the neighbor nodes at the present depth before moving on to nodes at the next depth level. It uses a queue data structure.

- **Properties/Identities:**
  - Finds the shortest path in terms of number of edges (unweighted graphs).
  - Guaranteed to visit all reachable vertices.
  - Time Complexity: $O(|V| + |E|)$ for adjacency list, $O(|V|^2)$ for adjacency matrix.
  - Space Complexity: $O(|V|)$ for queue and visited array.
- **Pitfalls/Tricks:**
  - Forgetting to mark nodes as visited to prevent cycles and redundant processing.
  - Using BFS for weighted shortest paths (Dijkstra's is needed).
- **Techniques/Shortcuts:**
  - Think of BFS as "level-by-level" exploration.
  - Useful for finding connected components, shortest path in unweighted graphs, and checking bipartiteness.

Bubble Sort

Definition: A simple sorting algorithm that repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order. The pass through the list is repeated until no swaps are needed, which indicates that the list is sorted.

- **Formulas/Theorems:**
  - **Worst-case Time Complexity:** $O(N^2)$ (e.g., reverse sorted array).
  - **Average-case Time Complexity:** $O(N^2)$.
  - **Best-case Time Complexity:** $O(N)$ (already sorted array, with optimization to stop if no swaps).
  - **Space Complexity:** $O(1)$ (in-place).
  - **Number of Swaps (Worst Case):** $N(N-1)/2$.
  - **Number of Comparisons (Worst Case):** $N(N-1)/2$.
- **Properties/Identities:**
  - Stable sorting algorithm.
  - Adaptive (can detect if an array is sorted and stop early).
- **Pitfalls/Tricks:**
  - Often used as a baseline for comparison due to its simplicity and inefficiency.
  - Not suitable for large datasets.

- **Techniques/Shortcuts:**
  - Understand its passes: after k passes, the last k elements are in their correct sorted positions.

Computer Science

Definition: The study of computation and information, including theoretical foundations, algorithmic design, and practical implementation. Algorithms are a core theoretical and practical component.

- **Properties/Identities:**
  - Algorithms are fundamental to all areas of Computer Science.

Depth First Search (DFS)

Definition: A graph traversal algorithm that explores as far as possible along each branch before backtracking. It uses a stack (explicit or implicit via recursion).

- **Properties/Identities:**
  - Can be used to find paths between two nodes, detect cycles, find connected components, and perform topological sorting.
  - Time Complexity: $O(|V| + |E|)$ for adjacency list, $O(|V|^2)$ for adjacency matrix.
  - Space Complexity: $O(|V|)$ for recursion stack or explicit stack, and visited array.
- **Pitfalls/Tricks:**
  - Can lead to very deep recursion for long paths, potentially causing stack overflow.
  - Does not guarantee shortest paths in unweighted graphs (BFS does).
- **Techniques/Shortcuts:**
  - Think of DFS as "going deep" before "going wide".
  - Useful for cycle detection, topological sort, and finding strongly connected components.

Dijkstra's Algorithm

Definition: A single-source shortest path algorithm for graphs with non-negative edge weights. It uses a greedy approach and a priority queue to efficiently find the shortest paths from a source vertex to all other vertices.

- **Formulas/Theorems:**
  - **Relaxation Step:** Same as Bellman-Ford.
  - **Time Complexity:**
    - With min-priority queue (binary heap): $O(|E| \log |V|)$ or $O(|E| + |V| \log |V|)$ if using Fibonacci heap.
    - With adjacency matrix (no priority queue, simple array scan): $O(|V|^2)$.
  - **Space Complexity:** $O(|V| + |E|)$ for graph, $O(|V|)$ for distances/predecessors.
- **Properties/Identities:**
  - Does not work correctly with negative edge weights.
  - Greedy algorithm.
  - Guaranteed to find the shortest path.
- **Pitfalls/Tricks:**
  - Applying it to graphs with negative edge weights (use Bellman-Ford instead).
  - Forgetting to update distances in the priority queue (decrease-key operation).
- **Techniques/Shortcuts:**
  - Always pick the unvisited vertex with the smallest known distance.
  - Visualize the "settling" of vertices.

Directed Acyclic Graph (DAG)

Definition: A directed graph that contains no directed cycles. This property makes them useful for representing processes with dependencies, like task scheduling.

- **Properties/Identities:**
  - Can always be topologically sorted.
  - No path can revisit a vertex.
  - Every finite DAG has at least one source (in-degree 0) and at least one sink (out-degree 0).
- **Pitfalls/Tricks:**
  - Confusing DAGs with general directed graphs that might have cycles.
  - Trying to apply algorithms that assume cycles (e.g., finding strongly connected components) without

modification.
- **Techniques/Shortcuts:**
  - Topological sort is a key algorithm for DAGs.
  - Dynamic programming problems on graphs often involve DAGs.

Double Hashing

Definition: A collision resolution technique in hashing that uses two hash functions. When a collision occurs with the first hash function, the second hash function is used to determine the step size for probing the hash table.

- **Formulas/Theorems:**
  - **Primary Hash Function:** $h_1(k)$.
  - **Secondary Hash Function:** $h_2(k)$. $h_2(k)$ must never return 0. Often $h_2(k) = R - (k \pmod R)$ for a prime $R < \text{table\_size}$.
  - **Probe Sequence:** $h(k, i) = (h_1(k) + i \cdot h_2(k)) \pmod M$, where M is table size and i is probe number (0, 1, 2, ...).
- **Properties/Identities:**
  - Minimizes clustering compared to linear probing.
  - Provides better distribution of keys.
  - Requires $h_2(k)$ to be relatively prime to M to ensure all slots are probed.
- **Pitfalls/Tricks:**
  - Choosing a secondary hash function that can return 0 or is not relatively prime to the table size, leading to incomplete probing.
  - Incorrectly implementing the modulo operation for negative results.
- **Techniques/Shortcuts:**
  - Ensure M is a prime number and R is a prime smaller than M for $h_2(k) = R - (k \pmod R)$.

Dynamic Programming

Definition: An algorithmic technique for solving complex problems by breaking them down into simpler subproblems. It's applicable when subproblems overlap and the problem exhibits optimal substructure. Solutions to subproblems are stored to avoid recomputation (memoization or tabulation).

- **Properties/Identities:**
  - **Optimal Substructure:** An optimal solution to the problem contains optimal solutions to its subproblems.
  - **Overlapping Subproblems:** The same subproblems are encountered multiple times.
  - **Memoization (Top-down):** Store results of subproblems as they are computed.
  - **Tabulation (Bottom-up):** Solve subproblems in a specific order, typically filling a table.
- **Pitfalls/Tricks:**
  - Incorrectly identifying the state of the DP problem.
  - Formulating an incorrect recurrence relation.
  - Forgetting base cases or handling them improperly.
- **Techniques/Shortcuts:**
  - Define the state: What information is needed to solve a subproblem?
  - Formulate the recurrence relation: How can a subproblem be solved using solutions to smaller subproblems?
  - Identify base cases.
  - Choose between memoization (recursive with caching) and tabulation (iterative).

Graph Algorithms

Definition: A category of algorithms designed to solve problems on graphs, which are mathematical structures used to model pairwise relations between objects. This includes traversal, shortest path, minimum spanning tree, and connectivity problems.

- **Properties/Identities:**
  - Common representations: Adjacency matrix, adjacency list.
  - Key problems: Shortest Path, MST, Graph Traversal, Connectivity.
- **Pitfalls/Tricks:**
  - Not considering graph type (directed/undirected, weighted/unweighted, cyclic/acyclic) when choosing an algorithm.
  - Memory limitations for dense graphs with adjacency matrices.
- **Techniques/Shortcuts:**

    - Always consider the graph representation that best suits the problem and its constraints.
    - Practice drawing graphs and tracing algorithms.

Graph Search

Definition: Algorithms for systematically visiting all vertices and edges in a graph. The two primary methods are Breadth-First Search (BFS) and Depth-First Search (DFS).

- **Properties/Identities:**
    - BFS: Level-by-level, uses queue, finds shortest path in unweighted graphs.
    - DFS: Explores deeply, uses stack/recursion, useful for cycle detection, topological sort, SCC.
- **Pitfalls/Tricks:**
    - Forgetting to use a 'visited' array/set to prevent infinite loops in cyclic graphs.
    - Choosing the wrong search algorithm for the problem (e.g., DFS for unweighted shortest path).
- **Techniques/Shortcuts:**
    - BFS is good for finding "closest" elements.
    - DFS is good for exploring "all paths" or "connectivity".

Greedy Algorithms

Definition: An algorithmic paradigm that makes the locally optimal choice at each stage with the hope of finding a global optimum. It doesn't always guarantee the globally optimal solution but works for specific problems.

- **Properties/Identities:**
    - **Greedy Choice Property:** A globally optimal solution can be reached by making a locally optimal (greedy) choice.
    - **Optimal Substructure:** An optimal solution to the problem contains optimal solutions to its subproblems.
- **Pitfalls/Tricks:**
    - Applying greedy approach to problems where it doesn't yield an optimal solution (e.g., general shortest path with negative weights, 0/1 Knapsack).
    - Not proving the greedy choice property and optimal substructure.
- **Techniques/Shortcuts:**
    - Common examples: Dijkstra's, Prim's, Kruskal's, Huffman Coding.
    - Try to prove correctness by contradiction or exchange argument.

Hashing

Definition: A technique used to map data of arbitrary size to fixed-size values (hash codes), typically used for efficient data retrieval in hash tables. Involves a hash function and collision resolution strategies.

- **Formulas/Theorems:**
    - **Hash Function:** $h(k) = k \pmod M$ (division method), $h(k) = \lfloor M(kA \pmod 1) \rfloor$ (multiplication method).
    - **Load Factor:** $\alpha = N/M$, where N is number of items, M is table size.
- **Properties/Identities:**
    - **Collision:** When two different keys map to the same hash value.
    - **Collision Resolution:** Open addressing (linear probing, quadratic probing, double hashing) or Separate chaining.
    - **Average Time Complexity (Search, Insert, Delete):** $O(1)$ (with good hash function and low load factor).
    - **Worst-case Time Complexity:** $O(N)$ (due to collisions).
- **Pitfalls/Tricks:**
    - Choosing a poor hash function that leads to many collisions (e.g., for string keys, summing ASCII values).
    - Ignoring the impact of load factor on performance.
- **Techniques/Shortcuts:**
    - For division method, choose M as a prime number not too close to a power of 2 or 10.
    - Understand the trade-offs between different collision resolution techniques.

Heap Sort

Definition: A comparison-based sorting algorithm that uses a binary heap data structure. It's an in-place algorithm that first builds a max-heap from the input data and then repeatedly extracts the maximum element and rebuilds the heap.

- **Formulas/Theorems:**

    - **Time Complexity:** $O(N \log N)$ for all cases (best, average, worst).
    - **Space Complexity:** $O(1)$ (in-place).
- **Properties/Identities:**
    - Not a stable sorting algorithm.
    - Uses a max-heap.
    - Build-heap step takes $O(N)$ time.
    - N extractions take $N \cdot O(\log N)$ time.
- **Pitfalls/Tricks:**
    - Confusing the heap property (min-heap vs. max-heap) with the sorting order.
    - Incorrectly implementing the `heapify` procedure.
- **Techniques/Shortcuts:**
    - Remember the two phases: build heap, then extract max and heapify.
    - The largest element is always at the root after heapify.

Huffman Code

Definition: A particular type of optimal prefix code used for lossless data compression. It's a greedy algorithm that builds a binary tree based on the frequencies of characters, assigning shorter codes to more frequent characters.

- **Properties/Identities:**
    - **Prefix Code:** No code is a prefix of another code, allowing unambiguous decoding.
    - **Optimal:** Produces the minimum possible expected code word length for a given set of character frequencies.
    - Uses a min-priority queue to repeatedly combine the two lowest-frequency nodes.
    - Time Complexity: $O(N \log N)$ where N is the number of unique characters.
- **Pitfalls/Tricks:**
    - Incorrectly building the Huffman tree (e.g., not using a min-priority queue, or incorrect combination logic).
    - Forgetting that it's a greedy algorithm.
- **Techniques/Shortcuts:**
    - Always combine the two nodes with the smallest frequencies.
    - The path from the root to a leaf defines the code word (e.g., left=0, right=1).

Identify Function

Definition: In a general mathematical or programming context, an identity function (often denoted id or I) is a function that always returns the same value that was used as its argument. $f(x) = x$. In algorithms, it might refer to a function used to uniquely identify elements or properties.

- **Properties/Identities:**
    - For any input x, $f(x) = x$.
    - It's the neutral element for function composition: $(f \circ id)(x) = f(x)$ and $(id \circ f)(x) = f(x)$.
- **Pitfalls/Tricks:**
    - This term is very generic; in an algorithmic context, it's usually implied or part of a larger concept (e.g., an identity hash function, or an identity matrix).

Insertion Sort

Definition: A simple sorting algorithm that builds the final sorted array (or list) one item at a time. It iterates through the input elements and removes one element at a time, finds the place where it belongs within the already sorted part, and inserts it there.

- **Formulas/Theorems:**
    - **Worst-case Time Complexity:** $O(N^2)$ (e.g., reverse sorted array).
    - **Average-case Time Complexity:** $O(N^2)$.
    - **Best-case Time Complexity:** $O(N)$ (already sorted array).
    - **Space Complexity:** $O(1)$ (in-place).
    - **Number of Swaps (Worst Case):** $N(N-1)/2$.
    - **Number of Comparisons (Worst Case):** $N(N-1)/2$.
- **Properties/Identities:**
    - Stable sorting algorithm.
    - Adaptive (efficient for nearly sorted arrays).
    - Good for small datasets.

- **Pitfalls/Tricks:**
  - Inefficient for large, unsorted arrays.
  - Understanding the "shift" operation correctly.
- **Techniques/Shortcuts:**
  - Think of it like sorting a hand of playing cards.
  - The first element is considered sorted.

Inversion

Definition: In an array A, an inversion is a pair of indices (i, j) such that $i < j$ and $A[i] > A[j]$. It measures how "unsorted" an array is.

- **Formulas/Theorems:**
  - **Maximum number of inversions in an array of size N:** $N(N-1)/2$ (for a reverse-sorted array).
  - **Minimum number of inversions:** 0 (for a sorted array).
- **Properties/Identities:**
  - The number of inversions can be counted efficiently using a modified Merge Sort algorithm in $O(N \log N)$ time.
- **Pitfalls/Tricks:**
  - Confusing inversions with just any pair of elements that are out of order; the index order $i < j$ is crucial.
- **Techniques/Shortcuts:**
  - To count inversions, adapt Merge Sort: when merging two sorted halves, if an element from the right half is taken before an element from the left half, it means all remaining elements in the left half form inversions with the taken element.

Linear Probing

Definition: A collision resolution technique in open addressing hashing where, upon a collision, the algorithm searches for the next available slot sequentially in the hash table (e.g., at $h(k)+1, h(k)+2, \ldots$).

- **Formulas/Theorems:**
  - **Probe Sequence:** $h(k, i) = (h(k) + i) \pmod M$, where M is table size and i is probe number (0, 1, 2, ...).
- **Properties/Identities:**
  - Simple to implement.
  - Suffers from primary clustering: long runs of occupied slots build up, increasing average search time.
- **Pitfalls/Tricks:**
  - Primary clustering significantly degrades performance, especially at high load factors.
  - Deletion is tricky: simply removing an element can break search chains. Often requires "lazy deletion" or re-hashing.
- **Techniques/Shortcuts:**
  - Understand that it's the simplest but often least efficient open addressing method.

Matrix Chain Ordering (Matrix Chain Multiplication)

Definition: A dynamic programming problem that seeks to find the most efficient way to multiply a sequence of matrices. The problem is not about performing the multiplications, but deciding the optimal parenthesization (order) to minimize the total number of scalar multiplications.

- **Formulas/Theorems:**
  - **Recurrence Relation:** Let $M[i, j]$ be the minimum number of scalar multiplications needed to compute the product $A_i A_{i+1} \ldots A_j$.

    $$M[i, j] = \min_{i \le k < j} (M[i, k] + M[k+1, j] + p_{i-1} p_k p_j)$$

    where A_i has dimensions $p_{i-1} \times p_i$.
  - **Base Case:** $M[i, i] = 0$ (single matrix requires no multiplications).
  - **Time Complexity:** $O(N^3)$ for N matrices.
  - **Space Complexity:** $O(N^2)$ for the DP table.
- **Properties/Identities:**
  - Exhibits optimal substructure and overlapping subproblems.
  - The order of multiplication matters for efficiency, not for the result.
- **Pitfalls/Tricks:**

    - Incorrectly setting up the dimensions array p. If there are N matrices $A_1, \ldots, A_N$, and A_i is $p_{i-1} \times p_i$, then the array p has $N+1$ elements.
    - Off-by-one errors in the recurrence relation indices.
- **Techniques/Shortcuts:**
    - Fill the DP table diagonally, starting with chain length 2, then 3, and so on.

Maximum Minimum

Definition: The problem of finding both the maximum and minimum elements in a given array or list. This can be done naively by iterating twice or more efficiently using a divide and conquer approach.

- **Formulas/Theorems:**
    - **Naive Approach (2N-2 comparisons):** Iterate once for max, once for min.
    - **Divide and Conquer Approach (approx. 3N/2 comparisons):**
        - If array size is 1, max=min=element.
        - If array size is 2, compare once.
        - Recursively find max/min in two halves, then compare the two maxes and two mins.
- **Properties/Identities:**
    - The divide and conquer approach is more efficient in terms of comparisons.
- **Pitfalls/Tricks:**
    - Forgetting to handle base cases (array of size 1 or 2) correctly in recursive solutions.
- **Techniques/Shortcuts:**
    - Pairwise comparison: process elements in pairs, comparing them. Then compare the smaller of the pair with the current min, and the larger with the current max. This also achieves approx. 3N/2 comparisons.

Merge Sort

Definition: A divide and conquer sorting algorithm that divides an unsorted list into N sublists, each containing one element (a list of one element is considered sorted), then repeatedly merges sublists to produce new sorted sublists until there is only one sorted list remaining.

- **Formulas/Theorems:**
    - **Recurrence Relation:** $T(N) = 2T(N/2) + O(N)$ (for dividing and merging).
    - **Time Complexity:** $O(N \log N)$ for all cases (best, average, worst).
    - **Space Complexity:** $O(N)$ (due to auxiliary array for merging).
- **Properties/Identities:**
    - Stable sorting algorithm.
    - Not an in-place algorithm (requires extra space).
    - Well-suited for external sorting.
- **Pitfalls/Tricks:**
    - Incorrectly implementing the merging step, which is crucial for correctness and efficiency.
    - Forgetting to copy remaining elements from one half if the other half is exhausted during merge.
- **Techniques/Shortcuts:**
    - The merge step is the core: combine two sorted arrays into one sorted array.
    - Can be used to count inversions.

Merging

Definition: The process of combining two or more sorted lists (or arrays) into a single sorted list. This is a fundamental operation in algorithms like Merge Sort.

- **Formulas/Theorems:**
    - **Time Complexity:** $O(N+M)$ to merge two sorted lists of sizes N and M.
    - **Space Complexity:** $O(N+M)$ for the new merged list.
- **Properties/Identities:**
    - Requires input lists to be sorted.
    - The output list is also sorted.
- **Pitfalls/Tricks:**
    - Off-by-one errors when handling array boundaries and indices.
    - Not correctly handling the case where one list is exhausted before the other.
- **Techniques/Shortcuts:**
    - Use two pointers, one for each input list, and a third pointer for the merged list.

Minimum Spanning Tree (MST)

Definition: For a connected, undirected, weighted graph, an MST is a subgraph that is a tree, connects all the vertices together, and has the minimum possible total edge weight. Algorithms include Prim's and Kruskal's.

- **Formulas/Theorems:**
  - **Cut Property:** For any cut (partition of vertices into two sets), if an edge crosses the cut and has strictly less weight than any other edge crossing the cut, then this edge must be in every MST.
  - **Cycle Property:** If an edge is the heaviest edge in any cycle of a graph, then it cannot be part of an MST.
  - An MST of a graph with $|V|$ vertices always has $|V| - 1$ edges.
- **Properties/Identities:**
  - Both Prim's and Kruskal's are greedy algorithms.
  - Prim's: Grows the MST from a starting vertex.
  - Kruskal's: Adds edges in increasing order of weight, avoiding cycles.
- **Pitfalls/Tricks:**
  - Applying MST algorithms to disconnected graphs (they will find an MST for each connected component, forming a Minimum Spanning Forest).
  - Confusing MST with shortest path problems.
- **Techniques/Shortcuts:**
  - Prim's uses a min-priority queue.
  - Kruskal's uses a Disjoint Set Union (DSU) data structure.

Number of Swap

Definition: A metric used to evaluate the efficiency of certain sorting algorithms, particularly comparison sorts. It counts how many times elements are exchanged during the sorting process.

- **Formulas/Theorems:**
  - **Bubble Sort (Worst Case):** $N(N-1)/2$.
  - **Selection Sort (Worst Case):** $N-1$.
  - **Insertion Sort (Worst Case):** $N(N-1)/2$.
  - **Quick Sort (Worst Case):** $O(N^2)$ swaps.
  - **Heap Sort (Worst Case):** $O(N \log N)$ swaps.
- **Properties/Identities:**
  - A lower number of swaps generally indicates better performance, especially when swap operations are costly.
  - Selection sort performs the minimum number of swaps among simple sorts.
- **Pitfalls/Tricks:**
  - Confusing swaps with comparisons. Both are important metrics but measure different aspects.
- **Techniques/Shortcuts:**
  - Memorize the worst-case swap counts for common sorting algorithms.

Prims Algorithm

Definition: A greedy algorithm that finds a Minimum Spanning Tree (MST) for a weighted undirected graph. It starts from an arbitrary vertex and grows the MST by adding the cheapest edge that connects a vertex in the MST to a vertex outside the MST.

- **Formulas/Theorems:**
  - **Time Complexity:**
    - With adjacency matrix (simple array scan): $O(|V|^2)$.
    - With adjacency list and binary min-priority queue: $O(|E| \log |V|)$ or $O(|E| + |V| \log |V|)$.
  - **Space Complexity:** $O(|V| + |E|)$ for graph, $O(|V|)$ for distances/parent array.
- **Properties/Identities:**
  - Greedy algorithm.
  - Builds the MST by adding vertices one by one.
  - Similar structure to Dijkstra's algorithm.
- **Pitfalls/Tricks:**
  - Forgetting to update edge weights in the priority queue when a shorter path to an unvisited vertex is found.
  - Applying to directed graphs (MST is for undirected graphs).
- **Techniques/Shortcuts:**
  - Maintain a set of vertices already in the MST and a min-priority queue of edges connecting to vertices outside

the MST.

Quick Sort

Definition: A highly efficient, in-place, divide and conquer sorting algorithm. It works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively.

- **Formulas/Theorems:**
  - **Worst-case Time Complexity:** $O(N^2)$ (e.g., already sorted array with first/last element as pivot).
  - **Average-case Time Complexity:** $O(N \log N)$.
  - **Best-case Time Complexity:** $O(N \log N)$.
  - **Space Complexity:** $O(\log N)$ (average, for recursion stack) or $O(N)$ (worst-case, for recursion stack).
  - **Recurrence Relation (Average Case):** $T(N) = T(k) + T(N-k-1) + O(N)$, where k is the size of one partition. For balanced partitions, $T(N) = 2T(N/2) + O(N)$.
- **Properties/Identities:**
  - Not a stable sorting algorithm.
  - In-place sorting algorithm.
  - Performance heavily depends on pivot selection.
- **Pitfalls/Tricks:**
  - Poor pivot selection can lead to $O(N^2)$ performance.
  - Incorrect partitioning logic can lead to infinite recursion or incorrect sorting.
- **Techniques/Shortcuts:**
  - Randomized pivot selection helps achieve average-case performance reliably.
  - Hoare's partition scheme is often more efficient than Lomuto's.

Recurrence Relation

Definition: An equation that recursively defines a sequence or function, where each term or value is given as a function of preceding terms. Used to describe the time complexity of recursive algorithms.

- **Formulas/Theorems:**
  - **Master Theorem:** For recurrences of the form $T(N) = aT(N/b) + f(N)$ where $a \geq 1, b > 1$.
    1. If $f(N) = O(N^{\log_b a - \epsilon})$ for some $\epsilon > 0$, then $T(N) = \Theta(N^{\log_b a})$.
    2. If $f(N) = \Theta(N^{\log_b a})$, then $T(N) = \Theta(N^{\log_b a} \log N)$.
    3. If $f(N) = \Omega(N^{\log_b a + \epsilon})$ for some $\epsilon > 0$, AND $af(N/b) \leq cf(N)$ for some $c < 1$ and large N, then $T(N) = \Theta(f(N))$.
- **Properties/Identities:**
  - Describes the growth rate of recursive algorithms.
  - Methods to solve: Substitution, Recursion Tree, Master Theorem.
- **Pitfalls/Tricks:**
  - Incorrectly applying the Master Theorem (e.g., when $f(N)$ doesn't fit any case or the regularity condition is not met).
  - Algebraic errors in substitution or recursion tree methods.
- **Techniques/Shortcuts:**
  - Memorize the three cases of the Master Theorem.
  - For simple recurrences, try expanding a few terms to find a pattern.

Recursion

Definition: A programming technique where a function calls itself directly or indirectly to solve a problem. It involves a base case (stopping condition) and a recursive step (reducing the problem to a smaller instance of itself).

- **Properties/Identities:**
  - **Base Case:** A condition that terminates the recursion.
  - **Recursive Step:** The function calls itself with a modified (smaller) input.
  - Often leads to elegant and concise code for problems with recursive structure.
- **Pitfalls/Tricks:**
  - Missing or incorrect base case leading to infinite recursion (stack overflow).
  - High space complexity due to recursion stack.
  - Performance overhead compared to iterative solutions due to function call stack management.

- **Techniques/Shortcuts:**
  - Always identify the base case first.
  - Ensure that each recursive call moves closer to the base case.
  - For some problems, recursion can be converted to iteration using a stack.

Searching

Definition: The process of finding a specific item (or items) within a collection of items. Common search algorithms include Linear Search and Binary Search.

- **Formulas/Theorems:**
  - **Linear Search (Worst Case):** $O(N)$ comparisons.
  - **Binary Search (Worst Case):** $O(\log N)$ comparisons.
- **Properties/Identities:**
  - Linear Search: Works on unsorted or sorted data.
  - Binary Search: Requires sorted data.
- **Pitfalls/Tricks:**
  - Using Linear Search on a large sorted array when Binary Search is applicable.
  - Errors in Binary Search boundary conditions.
- **Techniques/Shortcuts:**
  - Always check if data is sorted before choosing a search algorithm.

Selection Sort

Definition: A simple sorting algorithm that repeatedly finds the minimum element from the unsorted part of the list and swaps it with the element at the current position. It maintains two subarrays: sorted and unsorted.

- **Formulas/Theorems:**
  - **Worst-case Time Complexity:** $O(N^2)$.
  - **Average-case Time Complexity:** $O(N^2)$.
  - **Best-case Time Complexity:** $O(N^2)$.
  - **Space Complexity:** $O(1)$ (in-place).
  - **Number of Swaps (All Cases):** $N-1$ (minimum possible swaps for any comparison sort).
  - **Number of Comparisons (All Cases):** $N(N-1)/2$.
- **Properties/Identities:**
  - Not a stable sorting algorithm.
  - Performs the minimum number of swaps among simple sorts.
- **Pitfalls/Tricks:**
  - Its performance is consistently $O(N^2)$ regardless of input order, unlike Bubble Sort or Insertion Sort.
- **Techniques/Shortcuts:**
  - The key idea is to find the minimum and place it, then find the next minimum and place it, and so on.

Shortest Path

Definition: The problem of finding a path between two vertices (or from a source to all other vertices) in a graph such that the sum of the weights of its constituent edges is minimized. Algorithms include BFS (unweighted), Dijkstra's (non-negative weights), and Bellman-Ford (negative weights).

- **Properties/Identities:**
  - **Unweighted Graphs:** BFS finds shortest path in terms of number of edges.
  - **Non-negative Weighted Graphs:** Dijkstra's algorithm.
  - **Negative Weighted Graphs (no negative cycles):** Bellman-Ford algorithm.
  - **All-Pairs Shortest Path:** Floyd-Warshall algorithm.
- **Pitfalls/Tricks:**
  - Using the wrong algorithm for the given graph properties (e.g., Dijkstra's on negative weights).
  - Not handling disconnected components or unreachable vertices.
- **Techniques/Shortcuts:**
  - Always check for negative edge weights and negative cycles.
  - Understand the relaxation principle.

Sorting

Definition: The process of arranging elements of a list or array in a specific order (e.g., numerical, alphabetical, ascending, descending). It's a fundamental operation in computer science.

- **Formulas/Theorems:**
  - **Comparison Sort Lower Bound:** Any comparison-based sorting algorithm requires $\Omega(N \log N)$ comparisons in the worst case.
- **Properties/Identities:**
  - **Stable Sort:** Preserves the relative order of equal elements.
  - **In-place Sort:** Requires $O(1)$ or $O(\log N)$ auxiliary space.
  - **Adaptive Sort:** Performance improves for partially sorted input.
- **Pitfalls/Tricks:**
  - Confusing different sorting algorithm properties (stability, in-place, worst-case vs. average-case).
  - Choosing an inefficient sort for specific data characteristics.
- **Techniques/Shortcuts:**
  - Know the time and space complexities, stability, and in-place nature of common sorts.
  - For GATE, often questions involve comparing two sorts or analyzing a modified sort.

Space Complexity

Definition: A measure of the amount of memory an algorithm needs to run to completion. It includes the space required for input, output, and temporary variables, often expressed using asymptotic notation.

- **Properties/Identities:**
  - **Auxiliary Space:** The extra space used by the algorithm beyond the input data.
  - **In-place Algorithm:** An algorithm that transforms input using only a small, constant amount of auxiliary space, typically $O(1)$ or $O(\log N)$ for recursion stack.
- **Pitfalls/Tricks:**
  - Forgetting to account for the recursion stack space in recursive algorithms.
  - Confusing total space with auxiliary space.
- **Techniques/Shortcuts:**
  - Identify data structures created by the algorithm (arrays, queues, stacks, hash tables).
  - For recursive calls, the maximum depth of the recursion stack contributes to space complexity.

Strongly Connected Components (SCC)

Definition: In a directed graph, a strongly connected component (SCC) is a maximal subgraph such that for every pair of vertices u and v in the subgraph, there is a path from u to v and a path from v to u. Algorithms include Kosaraju's and Tarjan's.

- **Properties/Identities:**
  - SCCs partition the vertices of a directed graph.
  - If we contract each SCC into a single vertex, the resulting graph is a Directed Acyclic Graph (DAG).
  - Kosaraju's Algorithm: Two DFS passes (one on original graph, one on transpose graph).
  - Tarjan's Algorithm: One DFS pass, uses discovery times and low-link values.
- **Formulas/Theorems:**
  - **Time Complexity (Kosaraju's, Tarjan's):** $O(|V| + |E|)$.
- **Pitfalls/Tricks:**
  - Confusing SCCs with connected components in undirected graphs.
  - Incorrectly performing DFS on the transpose graph for Kosaraju's.
- **Techniques/Shortcuts:**
  - Kosaraju's: DFS on G to get finishing times, then DFS on G transpose in decreasing order of finishing times.
  - Tarjan's: Uses a stack and `disc` (discovery time) and `low` (lowest discovery time reachable) arrays.

Time Complexity

Definition: A measure of the amount of time an algorithm takes to run as a function of the length of its input. It's typically expressed using asymptotic notations (Big-O, Omega, Theta).

- **Properties/Identities:**
  - Focuses on the growth rate of operations as input size N increases.
  - Usually refers to worst-case time complexity, but average-case and best-case are also important.
  - Independent of machine specifics (CPU speed, memory access time).

- **Pitfalls/Tricks:**
  - Confusing constant factors with asymptotic growth.
  - Incorrectly analyzing loops or recursive calls.
  - Assuming best-case performance for general analysis.
- **Techniques/Shortcuts:**
  - Count dominant operations (comparisons, assignments, arithmetic operations).
  - For nested loops, multiply loop counts.
  - For recursive functions, use recurrence relations and Master Theorem.

Topological Sort

Definition: A linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every directed edge (u, v), vertex u comes before vertex v in the ordering. Not unique for all DAGs.

- **Properties/Identities:**
  - Only possible for DAGs.
  - Can be implemented using DFS or Kahn's algorithm (using in-degrees and a queue).
  - **Time Complexity:** $O(|V| + |E|)$.
- **Pitfalls/Tricks:**
  - Attempting to topologically sort a graph with cycles (it's impossible).
  - Incorrectly handling multiple possible topological sorts.
- **Techniques/Shortcuts:**
  - **DFS-based:** Perform DFS, then reverse the order of finishing times.
  - **Kahn's Algorithm:** Find all nodes with in-degree 0, add to queue. While queue not empty, dequeue, add to sorted list, decrement in-degree of neighbors. If neighbor's in-degree becomes 0, enqueue.

Tree Traversal

Definition: The process of visiting each node in a tree data structure exactly once. Common methods include Inorder, Preorder, Postorder (for binary trees), and Level-order traversal.

- **Properties/Identities:**
  - **Inorder (Left-Root-Right):** For BSTs, gives sorted elements.
  - **Preorder (Root-Left-Right):** Useful for creating a copy of the tree.
  - **Postorder (Left-Right-Root):** Useful for deleting a tree.
  - **Level-order (BFS-like):** Visits nodes level by level, uses a queue.
  - **Time Complexity:** $O(N)$ for a tree with N nodes.
  - **Space Complexity:** $O(H)$ for DFS-based (recursion stack, where H is height), $O(W)$ for BFS-based (queue, where W is max width).
- **Pitfalls/Tricks:**
  - Confusing the order of visits for different traversal types.
  - Incorrectly handling null nodes in recursive traversals.
- **Techniques/Shortcuts:**
  - Practice drawing the traversal paths on sample trees.
  - Remember the "Root" position in the name (Pre-Root-Order, In-Root-Order, Post-Root-Order).

Uniform Hashing

Definition: An idealized theoretical model for hashing where each key is equally likely to hash to any slot in the hash table, independent of where other keys hash. It implies a perfectly random distribution of keys.

- **Properties/Identities:**
  - Assumed for theoretical analysis of hashing algorithms (e.g., average case performance of open addressing).
  - Difficult to achieve in practice with real-world data.
  - Minimizes collisions and maximizes efficiency.
- **Pitfalls/Tricks:**
  - Assuming practical hash functions achieve uniform hashing, which is rarely true.
  - Not understanding that it's a theoretical ideal, not a practical hash function.
- **Techniques/Shortcuts:**
  - When analyzing hashing, remember that uniform hashing is the best-case scenario for collision distribution.

Quick Formula Reference

- **Asymptotic Notations:**
  - Big-O: $f(n) = O(g(n)) \implies \exists c, n_0 > 0 \text{ s.t. } 0 \le f(n) \le c \cdot g(n) \text{ for } n \ge n_0$.
  - Omega: $f(n) = \Omega(g(n)) \implies \exists c, n_0 > 0 \text{ s.t. } 0 \le c \cdot g(n) \le f(n) \text{ for } n \ge n_0$.
  - Theta: $f(n) = \Theta(g(n)) \implies \exists c_1, c_2, n_0 > 0 \text{ s.t. } 0 \le c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n) \text{ for } n \ge n_0$.
- **Binary Heap:**
  - Parent of i (0-indexed): $\lfloor (i-1)/2 \rfloor$.
  - Left child of i: $2i + 1$.
  - Right child of i: $2i + 2$.
  - Height: $\lfloor \log_2 N \rfloor$.
- **Binary Tree:**
  - Max nodes at level i: 2^i.
  - Max nodes in height h: $2^{h+1} - 1$.
  - Min height with N nodes: $\lceil \log_2(N+1) \rceil - 1$.
- **Hashing (Open Addressing):**
  - Linear Probing: $h(k, i) = (h(k) + i) \pmod M$.
  - Double Hashing: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \pmod M$.
  - Load Factor: $\alpha = N/M$.
- **Matrix Chain Ordering:**

$$M[i, j] = \min_{i \le k < j} (M[i, k] + M[k+1, j] + p_{i-1} p_k p_j)$$

  Base Case: $M[i, i] = 0$.
- **Recurrence Relation (Master Theorem):** $T(N) = aT(N/b) + f(N)$.
  1. If $f(N) = O(N^{\log_b a - \epsilon})$, $T(N) = \Theta(N^{\log_b a})$.
  2. If $f(N) = \Theta(N^{\log_b a})$, $T(N) = \Theta(N^{\log_b a} \log N)$.
  3. If $f(N) = \Omega(N^{\log_b a + \epsilon})$ and $af(N/b) \le cf(N)$, $T(N) = \Theta(f(N))$.
- **Sorting Algorithm Complexities:**

Algorithm	Time (Worst)	Time (Avg)	Space	Stable	In-place
Bubble Sort	$O(N^2)$	$O(N^2)$	$O(1)$	Yes	Yes
Insertion Sort	$O(N^2)$	$O(N^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(N^2)$	$O(N^2)$	$O(1)$	No	Yes
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N)$	Yes	No
Quick Sort	$O(N^2)$	$O(N \log N)$	$O(\log N)$	No	Yes
Heap Sort	$O(N \log N)$	$O(N \log N)$	$O(1)$	No	Yes

- **Graph Algorithm Complexities (Adjacency List):**
  - BFS: $O(|V| + |E|)$
  - DFS: $O(|V| + |E|)$
  - Dijkstra's (Binary Heap): $O(|E| \log |V|)$
  - Bellman-Ford: $O(|V| \cdot |E|)$
  - Prim's (Binary Heap): $O(|E| \log |V|)$
  - Kruskal's (DSU): $O(|E| \log |E|)$ or $O(|E| \log |V|)$
  - Topological Sort: $O(|V| + |E|)$
  - SCC (Kosaraju's/Tarjan's): $O(|V| + |E|)$

Important Tips for GATE

1. **Master Asymptotic Notations:** A significant portion of algorithm questions revolves around time and space complexity. Understand the definitions of $O, \Omega, \Theta, o, \omega$ thoroughly, and be adept at applying them to various code snippets and recurrence relations, especially using the Master Theorem.
2. **Understand Algorithm Design Paradigms:** Don't just memorize algorithms; understand *why* they work and which paradigm they belong to (Divide and Conquer, Greedy, Dynamic Programming). This helps in identifying the correct approach for new problems.
3. **Practice Recurrence Relations:** Solving recurrence relations is a frequently tested skill. Be comfortable with the substitution method, recursion tree method, and especially the Master Theorem. Pay attention to base cases and boundary conditions.
4. **Graph Algorithms are Crucial:** BFS, DFS, Dijkstra's, Bellman-Ford, Prim's, Kruskal's, Topological Sort, and SCCs are high-yield topics. Know their complexities, applications, and limitations (e.g., negative weights for

1.1 Algorithm Design (8)

✍ **Practice Test:** Test 1 (9Q)

1.1.1 Algorithm Design: GATE CSE 1992 | Question: 8

Let T be a Depth First Tree of a undirected graph G. An array P indexed by the vertices of G is given. $P[V]$ is the parent of vertex V, in T. Parent of the root is the root itself.

Give a method for finding and printing the cycle formed if the edge (u, v) of G not in T (i.e., $e \in G - T$) is now added to T.

Time taken by your method must be proportional to the length of the cycle.

Describe the algorithm in a PASCAL (C) – like language. Assume that the variables have been suitably declared.

gate1992 algorithms descriptive algorithm-design

Answer key

1.1.2 Algorithm Design: GATE CSE 1994 | Question: 7

An array A contains n integers in locations $A[0], A[1], \ldots A[n-1]$. It is required to shift the elements of the array cyclically to the left by K places, where $1 \leq K \leq n-1$. An incomplete algorithm for doing this in linear time, without using another array is given below. Complete the algorithm by filling in the blanks. Assume all variables are suitably declared.

```
min:=n;
i=0;
while _____ do
begin
   temp:=A[i];
   j:=i;
   while ____ do
   begin
      A[j]:=____;
      j:=(j+K) mod n;
     if j<min then
      min:=j;
   end;
   A[(n+i-K)mod n]:=____;
   i:=_____;
end;
```

gate1994 algorithms normal algorithm-design fill-in-the-blanks

Answer key

1.1.3 Algorithm Design: GATE CSE 2006 | Question: 17

An element in an array X is called a leader if it is greater than all elements to the right of it in X. The best algorithm to find all leaders in an array

A. solves it in linear time using a left to right pass of the array
B. solves it in linear time using a right to left pass of the array
C. solves it using divide and conquer in time $\Theta(n \log n)$
D. solves it in time $\Theta(n^2)$

gatecse-2006 algorithms normal algorithm-design

Answer key

1.1.4 Algorithm Design: GATE CSE 2006 | Question: 54

Given two arrays of numbers $a_1, \ldots, a_n$ and $b_1, \ldots, b_n$ where each number is 0 or 1, the fastest algorithm to find the largest span (i, j) such that $a_i + a_{i+1} + \cdots + a_j = b_i + b_{i+1} + \cdots + b_j$ or report that there is not such span,

A. Takes $O(3^n)$ and $\Omega(2^n)$ time if hashing is permitted
B. Takes $O(n^3)$ and $\Omega(n^{2.5})$ time in the key comparison mode
C. Takes $\Theta(n)$ time and space
D. Takes $O(\sqrt{n})$ time only if the sum of the $2n$ elements is an even number

gatecse-2006 algorithms normal algorithm-design time-complexity

Answer key

1.1.5 Algorithm Design: GATE CSE 2014 | Set 1 | Question: 37

There are 5 bags labeled 1 to 5. All the coins in a given bag have the same weight. Some bags have coins of weight 10 gm, others have coins of weight 11 gm. I pick $1, 2, 4, 8, 16$ coins respectively from bags 1 to 5 Their total weight comes out to 323 gm. Then the product of the labels of the bags having 11 gm coins is ___.

gatecse-2014-set1 algorithms numerical-answers normal algorithm-design

Answer key

1.1.6 Algorithm Design: GATE CSE 2019 | Question: 25

Consider a sequence of 14 elements: $A = [-5, -10, 6, 3, -1, -2, 13, 4, -9, -1, 4, 12, -3, 0]$. The sequence sum $S(i, j) = \Sigma_{k=i}^{j} A[k]$. Determine the maximum of $S(i, j)$, where $0 \leq i \leq j < 14$. (Divide and conquer approach may be used.)

Answer: _____

gatecse-2019 numerical-answers algorithms algorithm-design one-mark

Answer key

1.1.7 Algorithm Design: GATE CSE 2021 | Set 1 | Question: 40

Define R_n to be the maximum amount earned by cutting a rod of length n meters into one or more pieces of integer length and selling them. For $i > 0$, let $p[i]$ denote the selling price of a rod whose length is i meters. Consider the array of prices:

$$\mathrm{p}[1] = 1, \mathrm{p}[2] = 5, \mathrm{p}[3] = 8, \mathrm{p}[4] = 9, \mathrm{p}[5] = 10, \mathrm{p}[6] = 17, \mathrm{p}[7] = 18$$

Which of the following statements is/are correct about R_7?

A. $R_7 = 18$
B. $R_7 = 19$
C. R_7 is achieved by three different solutions
D. R_7 cannot be achieved by a solution consisting of three pieces

gatecse-2021-set1 multiple-selects algorithms algorithm-design two-marks

Answer key

1.1.8 Algorithm Design: GATE CSE 2024 | Set 2 | Question: 32

Consider an array X that contains n positive integers. A subarray of X is defined to be a sequence of array locations with consecutive indices.

The C code snippet given below has been written to compute the length of the longest subarray of X that contains at most two distinct integers. The code has two missing expressions labelled (P) and (Q).

```
int first=0, second=0, len1=0, len2=0, maxlen=0;
for (int i=0; i < n; i++) {
   if (X[i] == first) {
      len2++; len1++;
   } else if (X[i] == second) {
      len2++;
      len1 =     (P)     ;

      second = first;
```

```
    } else {
        len2 =      (Q)      ;

        len1 = 1; second = first;
    }
    if (len2 > maxlen) {
        maxlen = len2;
    }
    first = X[i];
}
```

Which one of the following options gives the CORRECT missing expressions?

(**Hint:** At the end of the i-th iteration, the value of $len1$ is the length of the longest subarray ending with $\mathrm{X}[i]$ that contains all equal values, and $len2$ is the length of the longest subarray ending with $\mathrm{X}[i]$ that contains at most two distinct values.)

A. $(\mathrm{P})\ \mathrm{len1}+1 \quad (\mathrm{Q})\ \mathrm{len2}+1$
B. $(\mathrm{P})\ 1 \quad (\mathrm{Q})\ \mathrm{len1}+1$
C. $(\mathrm{P})\ 1 \quad (\mathrm{Q})\ \mathrm{len2}+1$
D. $(\mathrm{P})\ \mathrm{len2}+1 \quad (\mathrm{Q})\ \mathrm{len1}+1$

gatecse-2024-set2 algorithms algorithm-design two-marks

Answer key

1.2 Algorithm Design Techniques (9)

1.2.1 Algorithm Design Techniques: GATE CSE 1990 | Question: 12b

Consider the following problem. Given n positive integers $a_1, a_2 \ldots a_n$, it is required to partition them in to two parts A and B such that, $\left|\sum_{i\in A} a_i - \sum_{i\in B} a_i\right|$ is minimised

Consider a greedy algorithm for solving this problem. The numbers are ordered so that $a_1 \geq a_2 \geq \ldots a_n$, and at i^{th} step, a_i is placed in that part whose sum in smaller at that step. Give an example with $n = 5$ for which the solution produced by the greedy algorithm is not optimal.

gate1990 descriptive algorithms algorithm-design-techniques

Answer key

1.2.2 Algorithm Design Techniques: GATE CSE 1990 | Question: 2-vii

Match the pairs in the following questions:

(a)	Strassen's matrix multiplication algorithm	(p)	Greedy method
(b)	Kruskal's minimum spanning tree algorithm	(q)	Dynamic programming
(c)	Biconnected components algorithm	(r)	Divide and Conquer
(d)	Floyd's shortest path algorithm	(s)	Depth-first search

gate1990 match-the-following algorithms algorithm-design-techniques easy

Answer key

1.2.3 Algorithm Design Techniques: GATE CSE 1994 | Question: 1.19, ISRO2016-31

Algorithm design technique used in quicksort algorithm is?

A. Dynamic programming
B. Backtracking
C. Divide and conquer
D. Greedy method

gate1994 algorithms algorithm-design-techniques quick-sort easy isro2016

Answer key

1.2.4 Algorithm Design Techniques: GATE CSE 1995 | Question: 1.5

Merge sort uses:

A. Divide and conquer strategy
B. Backtracking approach
C. Heuristic search
D. Greedy approach

gate1995 algorithms sorting easy algorithm-design-techniques merge-sort

Answer key

1.2.5 Algorithm Design Techniques: GATE CSE 1997 | Question: 1.5

The correct matching for the following pairs is

A.	All pairs shortest path	1.	Greedy
B.	Quick Sort	2.	Depth-First Search
C.	Minimum weight spanning tree	3.	Dynamic Programming
D.	Connected Components	4.	Divide and Conquer

A. A-2 B-4 C-1 D-3 B. A-3 B-4 C-1 D-2 C. A-3 B-4 C-2 D-1 D. A-4 B-1 C-2 D-3

gate1997 algorithms normal algorithm-design-techniques easy match-the-following

Answer key

1.2.6 Algorithm Design Techniques: GATE CSE 1998 | Question: 1.21, ISRO2008-16

Which one of the following algorithm design techniques is used in finding all pairs of shortest distances in a graph?

A. Dynamic programming
B. Backtracking
C. Greedy
D. Divide and Conquer

gate1998 algorithms algorithm-design-techniques easy isro2008

Answer key

1.2.7 Algorithm Design Techniques: GATE CSE 2015 | Set 1 | Question: 6

Match the following:

P.	Prim's algorithm for minimum spanning tree	i.	Backtracking
Q.	Floyd-Warshall algorithm for all pairs shortest path	ii.	Greedy method
R.	Merge sort	iii.	Dynamic programming
S.	Hamiltonian circuit	iv.	Divide and conquer

A. P-iii, Q-ii, R-iv, S-i
B. P-i, Q-ii, R-iv, S-iii
C. P-ii, Q-iii, R-iv, S-i
D. P-ii, Q-i, R-iii, S-iv

gatecse-2015-set1 algorithms normal match-the-following algorithm-design-techniques

Answer key

1.2.8 Algorithm Design Techniques: GATE CSE 2015 | Set 2 | Question: 36

Given below are some algorithms, and some algorithm design paradigms.

1.	Dijkstra's Shortest Path	i.	Divide and Conquer
2.	Floyd-Warshall algorithm to compute all pair shortest path	ii.	Dynamic Programming
3.	Binary search on a sorted array	iii.	Greedy design
4.	Backtracking search on a graph	iv.	Depth-first search
		v.	Breadth-first search

Match the above algorithms on the left to the corresponding design paradigm they follow.

A. 1-i, 2-iii, 3-i, 4-v
B. 1-iii, 2-iii, 3-i, 4-v
C. 1-iii, 2-ii, 3-i, 4-iv
D. 1-iii, 2-ii, 3-i, 4-v

gatecse-2015-set2 algorithms easy algorithm-design-techniques match-the-following

Answer key

1.2.9 Algorithm Design Techniques: GATE CSE 2017 | Set 1 | Question: 05

Consider the following table:

	Algorithms		Design Paradigms
(P)	Kruskal	(i)	Divide and Conquer
(Q)	Quicksort	(ii)	Greedy
(R)	Floyd-Warshall	(iii)	Dynamic Programming

Match the algorithms to the design paradigms they are based on.

A. $(P) \leftrightarrow (ii), (Q) \leftrightarrow (iii), (R) \leftrightarrow (i)$
B. $(P) \leftrightarrow (iii), (Q) \leftrightarrow (i), (R) \leftrightarrow (ii)$
C. $(P) \leftrightarrow (ii), (Q) \leftrightarrow (i), (R) \leftrightarrow (iii)$
D. $(P) \leftrightarrow (i), (Q) \leftrightarrow (ii), (R) \leftrightarrow (iii)$

gatecse-2017-set1 algorithms algorithm-design-techniques easy match-the-following

Answer key

1.3 Asymptotic Notations (22)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (15Q) Test 3 (15Q) Test 4 (4Q)

1.3.1 Asymptotic Notations: GATE CSE 1994 | Question: 1.23

Consider the following two functions:

$$g_1(n) = \begin{cases} n^3 \text{ for } 0 \leq n \leq 10,000 \\ n^2 \text{ for } n > 10,000 \end{cases}$$

$$g_2(n) = \begin{cases} n \text{ for } 0 \leq n \leq 100 \\ n^3 \text{ for } n > 100 \end{cases}$$

Which of the following is true?

A. $g_1(n)$ is $O(g_2(n))$
B. $g_1(n)$ is $O(n^3)$
C. $g_2(n)$ is $O(g_1(n))$
D. $g_2(n)$ is $O(n)$

gate1994 algorithms asymptotic-notations normal multiple-selects

Answer key

1.3.2 Asymptotic Notations: GATE CSE 1996 | Question: 1.11

Which of the following is false?

A. $100n \log n = O(\frac{n \log n}{100})$
B. $\sqrt{\log n} = O(\log \log n)$
C. If $0 < x < y$ then $n^x = O(n^y)$
D. $2^n \neq O(nk)$

gate1996 algorithms asymptotic-notations normal

Answer key

1.3.3 Asymptotic Notations: GATE CSE 1999 | Question: 2.21

If $T_1 = O(1)$, give the correct matching for the following pairs:

(M) $T_n = T_{n-1} + n$	(U) $T_n = O(n)$
(N) $T_n = T_{n/2} + n$	(V) $T_n = O(n \log n)$
(O) $T_n = T_{n/2} + n \log n$	(W) $T_n = O(n^2)$
(P) $T_n = T_{n-1} + \log n$	(X) $T_n = O(\log^2 n)$

A. M-W, N-V, O-U, P-X
B. M-W, N-U, O-X, P-V
C. M-V, N-W, O-X, P-U
D. M-W, N-U, O-V, P-X

gate1999 algorithms recurrence-relation asymptotic-notations normal match-the-following

Answer key

1.3.4 Asymptotic Notations: GATE CSE 2000 | Question: 2.17

Consider the following functions

- $f(n) = 3n^{\sqrt{n}}$
- $g(n) = 2^{\sqrt{n}\log_2 n}$
- $h(n) = n!$

Which of the following is true?

A. $h(n)$ is $O(f(n))$
B. $h(n)$ is $O(g(n))$
C. $g(n)$ is not $O(f(n))$
D. $f(n)$ is $O(g(n))$

gatecse-2000 algorithms asymptotic-notations normal

Answer key

1.3.5 Asymptotic Notations: GATE CSE 2001 | Question: 1.16

Let $f(n) = n^2 \log n$ and $g(n) = n(\log n)^{10}$ be two positive functions of n. Which of the following statements is correct?

A. $f(n) = O(g(n))$ and $g(n) \neq O(f(n))$
B. $g(n) = O(f(n))$ and $f(n) \neq O(g(n))$
C. $f(n) \neq O(g(n))$ and $g(n) \neq O(f(n))$
D. $f(n) = O(g(n))$ and $g(n) = O(f(n))$

gatecse-2001 algorithms asymptotic-notations time-complexity normal

Answer key

1.3.6 Asymptotic Notations: GATE CSE 2003 | Question: 20

Consider the following three claims:

I. $(n+k)^m = \Theta(n^m)$ where k and m are constants
II. $2^{n+1} = O(2^n)$
III. $2^{2n+1} = O(2^n)$

Which of the following claims are correct?

A. I and II
B. I and III
C. II and III
D. I, II, and III

gatecse-2003 algorithms asymptotic-notations normal

Answer key

1.3.7 Asymptotic Notations: GATE CSE 2004 | Question: 29

The tightest lower bound on the number of comparisons, in the worst case, for comparison-based sorting is of the order of

A. n
B. n^2
C. $n \log n$
D. $n \log^2 n$

gatecse-2004 algorithms sorting asymptotic-notations easy

Answer key

1.3.8 Asymptotic Notations: GATE CSE 2005 | Question: 37

Suppose $T(n) = 2T(\frac{n}{2}) + n, T(0) = T(1) = 1$
Which one of the following is FALSE?

A. $T(n) = O(n^2)$
B. $T(n) = \Theta(n \log n)$
C. $T(n) = \Omega(n^2)$
D. $T(n) = O(n \log n)$

gatecse-2005 algorithms asymptotic-notations recurrence-relation normal

Answer key

1.3.9 Asymptotic Notations: GATE CSE 2008 | Question: 39

Consider the following functions:

- $f(n) = 2^n$
- $g(n) = n!$
- $h(n) = n^{\log n}$

Which of the following statements about the asymptotic behavior of $f(n)$, $g(n)$ and $h(n)$ is true?

A. $f(n) = O(g(n)); g(n) = O(h(n))$
B. $f(n) = \Omega(g(n)); g(n) = O(h(n))$
C. $g(n) = O(f(n)); h(n) = O(f(n))$
D. $h(n) = O(f(n)); g(n) = \Omega(f(n))$

gatecse-2008 algorithms asymptotic-notations normal

Answer key

1.3.10 Asymptotic Notations: GATE CSE 2011 | Question: 37

Which of the given options provides the increasing order of asymptotic complexity of functions f_1, f_2, f_3 and f_4?

- $f_1(n) = 2^n$
- $f_2(n) = n^{3/2}$
- $f_3(n) = n \log_2 n$
- $f_4(n) = n^{\log_2 n}$

A. f_3, f_2, f_4, f_1
B. f_3, f_2, f_1, f_4
C. f_2, f_3, f_1, f_4
D. f_2, f_3, f_4, f_1

gatecse-2011 algorithms asymptotic-notations normal

Answer key

1.3.11 Asymptotic Notations: GATE CSE 2012 | Question: 18

Let $W(n)$ and $A(n)$ denote respectively, the worst case and average case running time of an algorithm executed on an input of size n. Which of the following is **ALWAYS TRUE**?

A. $A(n) = \Omega(W(n))$
B. $A(n) = \Theta(W(n))$
C. $A(n) = \mathrm{O}(W(n))$
D. $A(n) = \mathrm{o}(W(n))$

gatecse-2012 algorithms easy asymptotic-notations

Answer key

1.3.12 Asymptotic Notations: GATE CSE 2015 | Set 3 | Question: 4

Consider the equality $\sum_{i=0}^{n} i^3 = X$ and the following choices for X:

I. $\Theta(n^4)$

II. $\Theta(n^5)$
III. $O(n^5)$
IV. $\Omega(n^3)$

The equality above remains correct if X is replaced by

A. Only I
B. Only II
C. I or III or IV but not II
D. II or III or IV but not I

gatecse-2015-set3 algorithms asymptotic-notations normal

Answer key

1.3.13 Asymptotic Notations: GATE CSE 2015 | Set 3 | Question: 42

Let $f(n) = n$ and $g(n) = n^{(1+\sin\ n)}$, where n is a positive integer. Which of the following statements is/are correct?

I. $f(n) = O(g(n))$
II. $f(n) = \Omega(g(n))$

A. Only I
B. Only II
C. Both I and II
D. Neither I nor II

gatecse-2015-set3 algorithms asymptotic-notations normal

Answer key

1.3.14 Asymptotic Notations: GATE CSE 2017 | Set 1 | Question: 04

Consider the following functions from positive integers to real numbers:

$10, \sqrt{n}, n, \log_2 n, \frac{100}{n}$.
The CORRECT arrangement of the above functions in increasing order of asymptotic complexity is:

A. $\log_2 n, \frac{100}{n}, 10, \sqrt{n}, n$
B. $\frac{100}{n}, 10, \log_2 n, \sqrt{n}, n$
C. $10, \frac{100}{n}, \sqrt{n}, \log_2 n, n$
D. $\frac{100}{n}, \log_2 n, 10, \sqrt{n}, n$

gatecse-2017-set1 algorithms asymptotic-notations normal

Answer key

1.3.15 Asymptotic Notations: GATE CSE 2021 | Set 1 | Question: 3

Consider the following three functions.

$$f_1 = 10^n \quad f_2 = n^{\log n} \quad f_3 = n^{\sqrt{n}}$$

Which one of the following options arranges the functions in the increasing order of asymptotic growth rate?

A. f_3, f_2, f_1
B. f_2, f_1, f_3
C. f_1, f_2, f_3
D. f_2, f_3, f_1

gatecse-2021-set1 algorithms asymptotic-notations one-mark

Answer key

1.3.16 Asymptotic Notations: GATE CSE 2022 | Question: 1

Which one of the following statements is TRUE for all positive functions $f(n)$?

A. $f(n^2) = \theta(f(n)^2)$, when $f(n)$ is a polynomial
B. $f(n^2) = o(f(n)^2)$
C. $f(n^2) = O(f(n)^2)$, when $f(n)$ is an exponential function
D. $f(n^2) = \Omega(f(n)^2)$

gatecse-2022 algorithms asymptotic-notations one-mark

Answer key

1.3.17 Asymptotic Notations: GATE CSE 2023 | Question: 19

Let f and g be functions of natural numbers given by $f(n) = n$ and $g(n) = n^2$. Which of the following statements is/are TRUE?

A. $f \in O(g)$
B. $f \in \Omega(g)$
C. $f \in o(g)$
D. $f \in \Theta(g)$

gatecse-2023 algorithms asymptotic-notations multiple-selects one-mark

Answer key

1.3.18 Asymptotic Notations: GATE CSE 2023 | Question: 44

Consider functions Function_1 and Function_2 expressed in pseudocode as follows:

```
Function_1
  while n>1 do
    for i=1 to n do
      x = x + 1;
    end for
    n = ⌊n/2⌋;
  end while
```

```
Function_2
  for i = 1 to 100 * n do
    x = x + 1;
  end for
```

Let $f_1(n)$ and $f_2(n)$ denote the number of times the statement "$x = x + 1$" is executed in Function_1 and Function_2, respectively.

Which of the following statements is/are TRUE?

A. $f_1(n) \in \Theta(f_2(n))$
B. $f_1(n) \in o(f_2(n))$
C. $f_1(n) \in \omega(f_2(n))$
D. $f_1(n) \in O(n)$

gatecse-2023 algorithms asymptotic-notations multiple-selects two-marks

Answer key

1.3.19 Asymptotic Notations: GATE CSE 2024 | Set 2 | Question: 5

Let $\mathrm{T(n)}$ be the recurrence relation defined as follows:

$$T(0) = 1,$$
$$T(1) = 2, \text{ and}$$
$$T(n) = 5T(n-1) - 6T(n-2) \text{ for } n \geq 2$$

Which one of the following statements is TRUE?

A. $T(n) = \Theta(2^n)$
B. $T(n) = \Theta(n2^n)$
C. $T(n) = \Theta(3^n)$
D. $T(n) = \Theta(n3^n)$

gatecse-2024-set2 algorithms recurrence-relation asymptotic-notations one-mark

Answer key

1.3.20 Asymptotic Notations: GATE CSE 2026 | Set 2 | Question: 14

Consider the following functions, where n is a positive integer.

$$n^{1/3}, \log(n), \log(n!), 2^{\log(n)}$$

Which one of the following options lists the functions in increasing order of asymptotic growth rate?

Note: Assume the base of log to be 2.

A. $\log(n), n^{1/3}, 2^{\log(n)}, \log(n!)$
B. $n^{1/3}, \log(n), \log(n!), 2^{\log(n)}$

C. $\log(n), n^{1/3}, \log(n!), 2^{\log(n)}$
D. $2^{\log(n)}, n^{1/3}, \log(n), \log(n!)$

gatecse-2026-set2 algorithms asymptotic-notations one-mark

Answer key

1.3.21 Asymptotic Notations: GATE IT 2004 | Question: 55

Let $f(n)$, $g(n)$ and $h(n)$ be functions defined for positive integers such that $f(n) = O(g(n))$, $g(n) \neq O(f(n))$, $g(n) = O(h(n))$, and $h(n) = O(g(n))$.
Which one of the following statements is FALSE?

A. $f(n) + g(n) = O(h(n) + h(n))$
B. $f(n) = O(h(n))$
C. $h(n) \neq O(f(n))$
D. $f(n)h(n) \neq O(g(n)h(n))$

gateit-2004 algorithms asymptotic-notations normal

Answer key

1.3.22 Asymptotic Notations: GATE IT 2008 | Question: 10

Arrange the following functions in increasing asymptotic order:

a. $n^{1/3}$
b. e^n
c. $n^{7/4}$
d. $n \log^9 n$
e. 1.0000001^n

A. a, d, c, e, b
B. d, a, c, e, b
C. a, c, d, e, b
D. a, c, d, b, e

gateit-2008 algorithms asymptotic-notations normal

Answer key

1.4 Bellman Ford (2)

1.4.1 Bellman Ford: GATE CSE 2009 | Question: 13

Which of the following statement(s) is/are correct regarding Bellman-Ford shortest path algorithm?

P: Always finds a negative weighted cycle, if one exists.

Q: Finds whether any negative weighted cycle is reachable from the source.

A. P only
B. Q only
C. Both P and Q
D. Neither P nor Q

gatecse-2009 algorithms graph-algorithms normal bellman-ford

Answer key

1.4.2 Bellman Ford: GATE CSE 2013 | Question: 19

What is the time complexity of Bellman-Ford single-source shortest path algorithm on a complete graph of n vertices?

A. $\theta(n^2)$
B. $\theta(n^2 \log n)$
C. $\theta(n^3)$
D. $\theta(n^3 \log n)$

gatecse-2013 algorithms graph-algorithms normal bellman-ford

Answer key

1.5 Binary Heap (1)

1.5.1 Binary Heap: GATE CSE 2026 | Set 1 | Question: 13

Let n be an odd number greater than 100. Consider a binary minheap with n elements stored in an array P whose index starts from 1.

Which of the following indices of P do/does NOT correspond to any leaf node of the minheap?

A. $\frac{n+1}{2}$ B. $\frac{n-1}{2}$ C. $\frac{n-3}{2}$ D. n

gatecse-2026-set1 algorithms binary-heap multiple-selects one-mark

Answer key

1.6 Binary Search (4)

Practice Test: Test 1 (6Q)

1.6.1 Binary Search: GATE CSE 2021 | Set 2 | Question: 8

What is the worst-case number of arithmetic operations performed by recursive binary search on a sorted array of size n?

A. $\Theta(\sqrt{n})$
B. $\Theta(\log_2(n))$
C. $\Theta(n^2)$
D. $\Theta(n)$

gatecse-2021-set2 algorithms binary-search time-complexity one-mark

Answer key

1.6.2 Binary Search: GATE DA 2025 | Question: 17

For which of the following inputs does binary search take time $O(\log n)$ in the worst case?

A. An array of n integers in any order
B. A linked list of n integers in any order
C. An array of n integers in increasing order
D. A linked list of n integers in increasing order

gateda-2025 algorithms binary-search multiple-selects one-mark

Answer key

1.6.3 Binary Search: GATE DA 2026 | Question: 21

Let A be a sorted array containing 1000 distinct integers. You perform a recursive binary search on A to find an element y. Suppose each comparison checks whether the middle element computed during the current recursive step is equal to, less than, or greater than y.

The maximum number of comparisons that may have to be performed if y is not an element of A is ____. (Answer in integer)

gateda-2026 algorithms binary-search numerical-answers one-mark

Answer key

1.6.4 Binary Search: GATE DS&AI 2024 | Question: 30

Let $F(n)$ denote the maximum number of comparisons made while searching for an entry in a sorted array of size n using binary search.

Which ONE of the following options is TRUE?

A. $F(n) = F(\lfloor n/2 \rfloor) + 1$
B. $F(n) = F(\lfloor n/2 \rfloor) + F(\lceil n/2 \rceil)$
C. $F(n) = F(\lfloor n/2 \rfloor)$
D. $F(n) = F(n-1) + 1$

gate-ds-ai-2024 algorithms binary-search two-marks

Answer key

1.7 Binary Search Tree (3)

1.7.1 Binary Search Tree: GATE CSE 2026 | Set 1 | Question: 30

Let P be the set of all integers from 1 to 15. Consider any order of insertion of the elements of P into a binary search tree that creates a complete binary tree.

Which one of the following elements can NEVER be the third element that is inserted?

A. 4 B. 2 C. 10 D. 5

gatecse-2026-set1 algorithms binary-search-tree two-marks

Answer key

1.7.2 Binary Search Tree: GATE CSE 2026 | Set 1 | Question: 52

The following sequence corresponds to the preorder traversal of a binary search tree T :

$$50, 25, 13, 40, 30, 47, 75, 60, 70, 80, 77$$

The position of the element 60 in the postorder traversal of T is ____ . (answer in integer)

Note: The position begins with 1.

gatecse-2026-set1 numerical-answers two-marks binary-search-tree algorithms

Answer key

1.7.3 Binary Search Tree: GATE CSE 2026 | Set 2 | Question: 39

Consider a binary search tree (BST) with n leaf nodes $(n > 0)$. Given any node V, the key present in the node is denoted as $\text{Val}(V)$. All the keys present in the given BST are distinct. The keys belong to the set of real numbers.

For a node V, let $\text{Suc}(V)$ denote the node that is its inorder successor. If a node V does not have an inorder successor, then $\text{Suc}(V)$ is $NULL$. As there are no duplicates, if $\text{Suc}(V)$ is not $NULL$, then $\text{Val}(V) < \text{Val}(\text{Suc}(V))$.

Corresponding to every leaf node L_i that has a non-NULL $\text{Suc}(L_i)$, a new key k_i with the following property is to be inserted into the BST.

$$\text{Val}(L_i) < k_i < Val(\text{Suc}(L_i))$$

Let K represent the list of all such new keys to be inserted into the BST.

Which of the following statements is/are true?

A. K cannot have any duplicates
B. K will have at least one element
C. After inserting all keys from K, the height of the BST can increase at most by one
D. Number of nodes in the BST will double after inserting all keys from K

gatecse-2026-set2 algorithms binary-search-tree multiple-selects two-marks

Answer key

1.8 Binary Tree (1)

1.8.1 Binary Tree: GATE CSE 2026 | Set 1 | Question: 23

The height of a binary tree is the number of edges in the longest path from the root to a leaf in the tree. The maximum possible height of a full binary tree with 23 nodes is ____.(answer in integer)

gatecse-2026-set1 algorithms binary-tree numerical-answers one-mark

Answer key

1.9 Breadth First Search (3)

1.9.1 Breadth First Search: GATE CSE 2025 | Set 1 | Question: 33

Let $G(V, E)$ be an undirected and unweighted graph with 100 vertices. Let $d(u, v)$ denote the number of edges in a shortest path between vertices u and v in V. Let the maximum value of $d(u, v), u, v \in V$ such that $u \neq v$, be 30. Let T be any breadth-first-search tree of G. Which ONE of the given options is CORRECT for every such graph G ?

A. The height of T is exactly 15.
B. The height of T is exactly 30.
C. The height of T is at least 15.
D. The height of T is at least 30.

gatecse2025-set1 algorithms breadth-first-search shortest-path two-marks

Answer key

1.9.2 Breadth First Search: GATE DA 2026 | Question: 30

Consider a directed graph $G = (V, E)$, where V is the finite set of vertices and E is the set of directed edges between the vertices. G may contain cycles but there is no self-loop. Further, G may not be strongly connected.

Let G^R be the graph obtained by reversing the directions of all the edges in G without changing the set of vertices. Assume that Breadth First Search (BFS) or Depth First Search (DFS) from any given vertex v of a graph visits only the reachable vertices from v in that graph.

Which of the following statements must always be true, regardless of the structure of G?

A. If u is a reachable vertex in the BFS of G^R from v, then u is also a reachable vertex in the DFS of G from v.
B. In G^R, the BFS traversal from v will visit exactly the same set of vertices as the DFS from v in G.
C. The order of vertices visited in the BFS of G^R from v is the reverse of the order of vertices visited in the DFS of G from v.
D. If u is a reachable vertex in the DFS of G from v, then v is also a reachable vertex in the BFS of G^R from u.

gateda-2026 algorithms graph-algorithms breadth-first-search depth-first-search two-marks

1.9.3 Breadth First Search: GATE DS&AI 2024 | Question: 34

Consider a state space where the start state is number 1. The successor function for the state numbered n returns two states numbered $n + 1$ and $n + 2$. Assume that the states in the unexpanded state list are expanded in the ascending order of numbers and the previously expanded states are not added to the unexpanded state list.

Which ONE of the following statements about breadth-first search (BFS) and depth-first search (DFS) is true, when reaching the goal state number 6?

A. BFS expands more states than DFS.
B. DFS expands more states than BFS.
C. Both BFS and DFS expand equal number of states.
D. Both BFS and DFS do not reach the goal state number 6.

gate-ds-ai-2024 algorithms breadth-first-search depth-first-search two-marks

Answer key

1.10 Bubble Sort (1)

1.10.1 Bubble Sort: GATE CSE 1995 | Question: 12

Consider the following sequence of numbers:

$$92, 37, 52, 12, 11, 25$$

Use Bubble sort to arrange the sequence in ascending order. Give the sequence at the end of each of the first five passes.

gate1995 algorithms sorting easy descriptive bubble-sort

Answer key

1.11 Computer Science (1)

1.11.1 Computer Science: GATE CS Practice : The "Master Theorem" (Algorithms)

Consider the following recurrence relation describing the running time of an algorithm:

$$T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}$$

$$(Base\ condition: T(1) = 1)$$

What is the asymptotic tight bound $\Theta(T(n))$?

A. $\Theta(n)$

B. $\Theta(n \log n)$

C. $\Theta(n \log \log n)$

D. $\Theta(n(\log n)^2)$

algorithms master-theorem recurrence-relation computer-science gate-preparation

Answer key

1.12 Depth First Search (2)

✍ **Practice Test:** Test 1 (12Q)

1.12.1 Depth First Search: GATE CSE 2026 | Set 1 | Question: 40

Consider the following pseudocode for depth-first search (DFS) algorithm which takes a directed graph $G(V, E)$ as input, where $d[v]$ and $f[v]$ are the discovery time and finishing time, respectively, of the vertex $v \in V$.

```
DFS(G):
    unmark all v ∈ V
    t ← 0
    for each v ∈ V
        if v is unmarked
            t ← Explore(G, v, t)
        end if
    end for
```

```
Explore(G, v, t):
    mark v
    t ← t + 1
    d[v] ← t
    for each (v, w) ∈ E
        if w is unmarked
            t ← Explore(G, w, t)
        end if
    end for
    t ← t + 1
    f[v] ← t
    return t
```

Suppose that the input directed graph $G(V, E)$ is a directed acyclic graph (DAG).
For an edge $(u, v) \in E$, which of the following options will NEVER be correct?

A. $d[u] < d[v] < f[v] < f[u]$

B. $d[v] < d[u] < f[u] < f[v]$

C. $d[v] < f[v] < d[u] < f[u]$

D. $d[u] < d[v] < f[u] < f[v]$

gatecse-2026-set1 algorithms depth-first-search two-marks multiple-selects

Answer key

1.12.2 Depth First Search: GATE DA 2025 | Question: 55

Consider a directed graph $G = (V, E)$, where $V = \{0, 1, 2, \ldots, 100\}$ and $E = \{(i, j) : 0 < j - i \leq 2, \text{ for all } i, j \in V\}$. Suppose the adjacency list of each vertex is in decreasing order of vertex number, and depth-first search (DFS) is performed at vertex 0. The number of vertices that will be discovered after vertex 50 is _____ (Answer in integer)

gateda-2025 algorithms depth-first-search numerical-answers two-marks

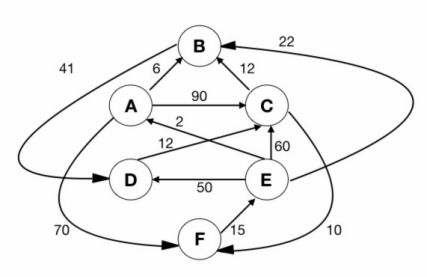
Answer key

1.13 Dijkstras Algorithm (5)

Practice Test: Test 1 (12Q)

1.13.1 Dijkstras Algorithm: GATE CSE 1996 | Question: 17

Let G be the directed, weighted graph shown in below figure



We are interested in the shortest paths from A.

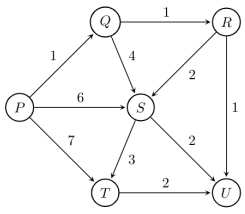
a. Output the sequence of vertices identified by the Dijkstra's algorithm for single source shortest path when the algorithm is started at node A
b. Write down sequence of vertices in the shortest path from A to E
c. What is the cost of the shortest path from A to E?

gate1996 algorithms graph-algorithms normal dijkstras-algorithm descriptive

Answer key

1.13.2 Dijkstras Algorithm: GATE CSE 2004 | Question: 44

Suppose we run Dijkstra's single source shortest path algorithm on the following edge-weighted directed graph with vertex P as the source.



In what order do the nodes get included into the set of vertices for which the shortest path distances are finalized?

A. P, Q, R, S, T, U B. P, Q, R, U, S, T C. P, Q, R, U, T, S D. P, Q, T, R, U, S

gatecse-2004 algorithms graph-algorithms normal dijkstras-algorithm

Answer key

1.13.3 Dijkstras Algorithm: GATE CSE 2005 | Question: 38

Let $G(V, E)$ be an undirected graph with positive edge weights. Dijkstra's single source shortest path algorithm can be implemented using the binary heap data structure with time complexity:

A. $O\left(|V|^2\right)$ B. $O(|E| + |V| \log |V|)$

C. $O(|V|\log|V|)$

D. $O((|E|+|V|)\log|V|)$

gatecse-2005 algorithms graph-algorithms normal dijkstras-algorithm

Answer key

1.13.4 Dijkstras Algorithm: GATE CSE 2006 | Question: 12

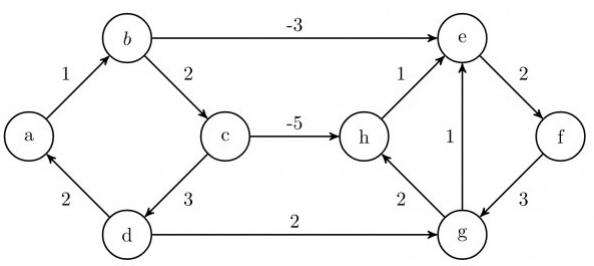
To implement Dijkstra's shortest path algorithm on unweighted graphs so that it runs in linear time, the data structure to be used is:

A. Queue B. Stack C. Heap D. B-Tree

gatecse-2006 algorithms graph-algorithms easy dijkstras-algorithm

Answer key

1.13.5 Dijkstras Algorithm: GATE CSE 2008 | Question: 45



Dijkstra's single source shortest path algorithm when run from vertex a in the above graph, computes the correct shortest path distance to

A. only vertex a

B. only vertices a, e, f, g, h

C. only vertices a, b, c, d

D. all the vertices

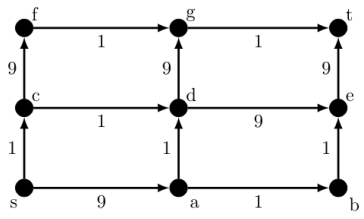
gatecse-2008 algorithms graph-algorithms normal dijkstras-algorithm

Answer key

1.14 Directed Acyclic Graph (2)

1.14.1 Directed Acyclic Graph: GATE CSE 2021 | Set 2 | Question: 55

In a directed acyclic graph with a source vertex **s**, the $quality\text{-}score$ of a directed path is defined to be the product of the weights of the edges on the path. Further, for a vertex v other than **s**, the quality-score of v is defined to be the maximum among the quality-scores of all the paths from **s** to v. The quality-score of **s** is assumed to be 1.



The sum of the quality-scores of all vertices on the graph shown above is _____

gatecse-2021-set2 algorithms graph-algorithms directed-acyclic-graph numerical-answers two-marks

Answer key

1.14.2 Directed Acyclic Graph: GATE CSE 2026 | Set 2 | Question: 27

Let G be a weighted directed acyclic graph with m edges and n vertices. Given G and a source vertex s in G, which one of the following options gives the worst case time complexity of the fastest algorithm to find the lengths of shortest paths from s to all vertices that are reachable from s in G?

A. $\Theta(m+n)$

B. $\Theta(m+n\log(n))$

C. $\Theta(nm)$ D. $\Theta(n^3)$

gatecse-2026-set2 algorithms shortest-path directed-acyclic-graph time-complexity two-marks

Answer key

1.15 Double Hashing (2)

1.15.1 Double Hashing: GATE CSE 2020 | Question: 23

Consider a double hashing scheme in which the primary hash function is $h_1(k) = k \bmod 23$, and the secondary hash function is $h_2(k) = 1 + (k \bmod 19)$. Assume that the table size is 23. Then the address returned by probe 1 in the probe sequence (assume that the probe sequence begins at probe 0) for key value $k = 90$ is_____.

gatecse-2020 numerical-answers algorithms hashing one-mark double-hashing

Answer key

1.15.2 Double Hashing: GATE CSE 2025 | Set 1 | Question: 55

In a double hashing scheme, $h_1(k) = k \bmod 11$ and $h_2(k) = 1 + (k \bmod 7)$ are the auxiliary hash functions. The size m of the hash table is 11. The hash function for the i-th probe in the open address table is $[h_1(k) + ih_2(k)] \bmod m$. The following keys are inserted in the given order: $63, 50, 25, 79, 67, 24$.

The slot at which key 24 gets stored is _____ . (Answer in integer)

gatecse2025-set1 algorithms hashing double-hashing numerical-answers easy two-marks

Answer key

1.16 Dynamic Programming (10)

✍ **Practice Test:** Test 1 (13Q)

1.16.1 Dynamic Programming: GATE CSE 2008 | Question: 80

The subset-sum problem is defined as follows. Given a set of n positive integers, $S = \{a_1, a_2, a_3, \ldots, a_n\}$, and positive integer W, is there a subset of S whose elements sum to W? A dynamic program for solving this problem uses a 2-dimensional Boolean array, X, with n rows and $W + 1$ columns. $X[i, j], 1 \leq i \leq n, 0 \leq j \leq W$, is TRUE, if and only if there is a subset of $\{a_1, a_2, \ldots, a_i\}$ whose elements sum to j.

Which of the following is valid for $2 \leq i \leq n$, and $a_i \leq j \leq W$?

A. $X[i, j] = X[i-1, j] \vee X[i, j-a_i]$
B. $X[i, j] = X[i-1, j] \vee X[i-1, j-a_i]$
C. $X[i, j] = X[i-1, j] \wedge X[i, j-a_i]$
D. $X[i, j] = X[i-1, j] \wedge X[i-1, j-a_i]$

gatecse-2008 algorithms difficult dynamic-programming

Answer key

1.16.2 Dynamic Programming: GATE CSE 2008 | Question: 81

The subset-sum problem is defined as follows. Given a set of n positive integers, $S = \{a_1, a_2, a_3, \ldots, a_n\}$, and positive integer W, is there a subset of S whose elements sum to W? A dynamic program for solving this problem uses a 2-dimensional Boolean array, X, with n rows and $W + 1$ columns. $X[i, j], 1 \leq i \leq n, 0 \leq j \leq W$, is TRUE, if and only if there is a subset of $\{a_1, a_2, \ldots, a_i\}$ whose elements sum to j.

Which entry of the array X, if TRUE, implies that there is a subset whose elements sum to W?

A. $X[1, W]$ B. $X[n, 0]$ C. $X[n, W]$ D. $X[n-1, n]$

gatecse-2008 algorithms normal dynamic-programming

Answer key

1.16.3 Dynamic Programming: GATE CSE 2009 | Question: 53

A sub-sequence of a given sequence is just the given sequence with some elements (possibly none or all) left out. We are given two sequences $X[m]$ and $Y[n]$ of lengths m and n, respectively with indexes of X and Y starting from 0.

We wish to find the length of the longest common sub-sequence (LCS) of $X[m]$ and $Y[n]$ as $l(m,n)$, where an incomplete recursive definition for the function $I(i,j)$ to compute the length of the LCS of $X[m]$ and $Y[n]$ is given below:

l(i,j) = 0, if either i = 0 or j = 0
= expr1, if i,j > 0 and X[i-1] = Y[j-1]
= expr2, if i,j > 0 and X[i-1] ≠ Y[j-1]

Which one of the following options is correct?

A. $\text{expr1} = l(i-1,j)+1$
B. $\text{expr1} = l(i,j-1)$
C. $\text{expr2} = \max(l(i-1,j), l(i,j-1))$
D. $\text{expr2} = \max(l(i-1,j-1), l(i,j))$

gatecse-2009 algorithms normal dynamic-programming recursion

Answer key

1.16.4 Dynamic Programming: GATE CSE 2009 | Question: 54

A sub-sequence of a given sequence is just the given sequence with some elements (possibly none or all) left out. We are given two sequences $X[m]$ and $Y[n]$ of lengths m and n, respectively with indexes of X and Y starting from 0.

We wish to find the length of the longest common sub-sequence (LCS) of $X[m]$ and $Y[n]$ as $l(m,n)$, where an incomplete recursive definition for the function $I(i,j)$ to compute the length of the LCS of $X[m]$ and $Y[n]$ is given below:

l(i,j) = 0, if either i = 0 or j = 0
= expr1, if i,j > 0 and X[i-1] = Y[j-1]
= expr2, if i,j > 0 and X[i-1] ≠ Y[j-1]

The value of $l(i,j)$ could be obtained by dynamic programming based on the correct recursive definition of $l(i,j)$ of the form given above, using an array $L[M,N]$, where $M = m+1$ and $N = n+1$, such that $L[i,j] = l(i,j)$.

Which one of the following statements would be TRUE regarding the dynamic programming solution for the recursive definition of $l(i,j)$?

A. All elements of L should be initialized to 0 for the values of $l(i,j)$ to be properly computed.
B. The values of $l(i,j)$ may be computed in a row major order or column major order of $L[M,N]$.
C. The values of $l(i,j)$ cannot be computed in either row major order or column major order of $L[M,N]$.
D. $L[p,q]$ needs to be computed before $L[r,s]$ if either $p < r$ or $q < s$.

gatecse-2009 normal algorithms dynamic-programming recursion

Answer key

1.16.5 Dynamic Programming: GATE CSE 2010 | Question: 34

The weight of a sequence $a_0, a_1, \ldots, a_{n-1}$ of real numbers is defined as $a_0 + a_1/2 + \cdots + a_{n-1}/2^{n-1}$. A subsequence of a sequence is obtained by deleting some elements from the sequence, keeping the order of the remaining elements the same. Let X denote the maximum possible weight of a subsequence of $a_o, a_1, \ldots, a_{n-1}$ and Y the maximum possible weight of a subsequence of $a_1, a_2, \ldots, a_{n-1}$. Then X is equal to

A. $max(Y, a_0 + Y)$
B. $max(Y, a_0 + Y/2)$
C. $max(Y, a_0 + 2Y)$
D. $a_0 + Y/2$

gatecse-2010 algorithms dynamic-programming normal

Answer key

1.16.6 Dynamic Programming: GATE CSE 2011 | Question: 25

An algorithm to find the length of the longest monotonically increasing sequence of numbers in an array $A[0:n-1]$ is given below.

Let L_i, denote the length of the longest monotonically increasing sequence starting at index i in the array.

Initialize $L_{n-1} = 1$.

For all i such that $0 \leq i \leq n-2$

$$L_i = \begin{cases} 1 + L_{i+1} & \text{if A[i]} < \text{A[i+1]} \\ 1 & \text{Otherwise} \end{cases}$$

Finally, the length of the longest monotonically increasing sequence is $\max(L_0, L_1, \ldots, L_{n-1})$.

Which of the following statements is **TRUE**?

A. The algorithm uses dynamic programming paradigm
B. The algorithm has a linear complexity and uses branch and bound paradigm
C. The algorithm has a non-linear polynomial complexity and uses branch and bound paradigm
D. The algorithm uses divide and conquer paradigm

gatecse-2011 algorithms easy dynamic-programming

Answer key

1.16.7 Dynamic Programming: GATE CSE 2014 | Set 2 | Question: 37

Consider two strings A = *"qpqrr"* and B = *"pqprqrp"*. Let x be the length of the longest common subsequence *(not necessarily contiguous)* between A and B and let y be the number of such longest common subsequences between A and B. Then $x + 10y =$ ___.

gatecse-2014-set2 algorithms normal numerical-answers dynamic-programming

Answer key

1.16.8 Dynamic Programming: GATE CSE 2014 | Set 3 | Question: 37

Suppose you want to move from 0 to 100 on the number line. In each step, you either move right by a unit distance or you take a *shortcut*. A shortcut is simply a pre-specified pair of integers $i, j \text{ with } i < j$. Given a shortcut (i, j), if you are at position i on the number line, you may directly move to j. Suppose $T(k)$ denotes the smallest number of steps needed to move from k to 100. Suppose further that there is at most 1 shortcut involving any number, and in particular, from 9 there is a shortcut to 15. Let y and z be such that $T(9) = 1 + \min(T(y), T(z))$. Then the value of the product yz is _____.

gatecse-2014-set3 algorithms normal numerical-answers dynamic-programming

Answer key

1.16.9 Dynamic Programming: GATE CSE 2016 | Set 2 | Question: 14

The Floyd-Warshall algorithm for all-pair shortest paths computation is based on

A. Greedy paradigm.
B. Divide-and-conquer paradigm.
C. Dynamic Programming paradigm.
D. Neither Greedy nor Divide-and-Conquer nor Dynamic Programming paradigm.

gatecse-2016-set2 algorithms dynamic-programming easy

Answer key

1.16.10 Dynamic Programming: GATE CSE 2026 | Set 2 | Question: 29

Consider a table T, where the elements $T[i][j], 0 \leq i, j \leq n$, represent the cost of the optimal solutions of different subproblems of a problem that is being solved using a dynamic programming algorithm. The recursive formulation to compute the table entries is as follows:

$$T[0][k] = T[k][0] = 1 \qquad \text{for } k = 0, 1, 2, \ldots, n$$
$$T[i][j] = 2T[i-1][j] + 3T[i][j-1] \qquad \text{for } 1 \leq i, j \leq n$$

Consider the following two algorithms to compute entries of T. Assume that for both the algorithms, for all $0 \leq i, j \leq n, T[i][j]$ has been initialized to 1.
Algorithm B_1 : For $i = 1, 2, \ldots, n$

$$\begin{aligned}&\text{For } j = 1, 2, \ldots, n\\ &\quad T[i][j] = 2T[i-1][j] + 3T[i][j-1]\end{aligned}$$

Algorithm B_2 : For $s = 2, 3, \ldots, 2n$

$$\begin{aligned}&\text{For } i = 1, 2, \ldots, n\\ &\quad \text{For } j = 1, 2, \ldots, n\\ &\quad\quad \text{If } (i + j == s)\\ &\quad\quad\quad T[i][j] = 2T[i-1][j] + 3T[i][j-1]\end{aligned}$$

Algorithm $B_k, k \in \{1, 2\}$ is said to be correct if and only if it calculates the correct values of $T[i][j]$, for all $0 \leq i, j \leq n$, (as per the recursive formulation) at the end of the execution of the algorithm B_k.

Which one of the following statements is true?

A. Both algorithms B_1 and B_2 are correct
B. Algorithm B_1 is correct, but algorithm B_2 is incorrect
C. Algorithm B_2 is correct, but algorithm B_1 is incorrect
D. Both algorithms B_1 and B_2 are incorrect

gatecse-2026-set2 algorithms dynamic-programming two-marks

Answer key

1.17 Graph Algorithms (11)

Practice Test: Test 1 (14Q)

1.17.1 Graph Algorithms: GATE CSE 1994 | Question: 1.22

Which of the following statements is false?

A. Optimal binary search tree construction can be performed efficiently using dynamic programming
B. Breadth-first search cannot be used to find connected components of a graph
C. Given the prefix and postfix walks over a binary tree, the binary tree cannot be uniquely constructed.
D. Depth-first search can be used to find connected components of a graph

gate1994 algorithms normal graph-algorithms

Answer key

1.17.2 Graph Algorithms: GATE CSE 2003 | Question: 70

Let $G = (V, E)$ be a directed graph with n vertices. A path from v_i to v_j in G is a sequence of vertices ($v_i, v_{i+1}, \ldots, v_j$) such that $(v_k, v_{k+1}) \in E$ for all k in i through $j - 1$. A simple path is a path in which no vertex appears more than once.

Let A be an $n \times n$ array initialized as follows:

$$A[j,k] = \begin{cases} 1, & \text{if } (j,k) \in E \\ 0, & \text{otherwise} \end{cases}$$

Consider the following algorithm:

```
for i=1 to n
   for j=1 to n
      for k=1 to n
         A[j,k] = max(A[j,k], A[j,i] + A[i,k]);
```

Which of the following statements is necessarily true for all j and k after termination of the above algorithm?

A. $A[j,k] \leq n$
B. If $A[j,j] \geq n-1$ then G has a Hamiltonian cycle
C. If there exists a path from j to k, $A[j,k]$ contains the longest path length from j to k
D. If there exists a path from j to k, every simple path from j to k contains at most $A[j,k]$ edges

gatecse-2003 algorithms graph-algorithms normal

Answer key

1.17.3 Graph Algorithms: GATE CSE 2005 | Question: 82a

Let s and t be two vertices in a undirected graph $G = (V, E)$ having distinct positive edge weights. Let $[X, Y]$ be a partition of V such that $s \in X$ and $t \in Y$. Consider the edge e having the minimum weight amongst all those edges that have one vertex in X and one vertex in Y.

The edge e must definitely belong to:

A. the minimum weighted spanning tree of G
B. the weighted shortest path from s to t
C. each path from s to t
D. the weighted longest path from s to t

gatecse-2005 algorithms graph-algorithms normal

Answer key

1.17.4 Graph Algorithms: GATE CSE 2005 | Question: 82b

Let s and t be two vertices in a undirected graph $G = (V, E)$ having distinct positive edge weights. Let $[X, Y]$ be a partition of V such that $s \in X$ and $t \in Y$. Consider the edge e having the minimum weight amongst all those edges that have one vertex in X and one vertex in Y.

Let the weight of an edge e denote the congestion on that edge. The congestion on a path is defined to be the maximum of the congestions on the edges of the path. We wish to find the path from s to t having minimum congestion. Which of the following paths is always such a path of minimum congestion?

A. a path from s to t in the minimum weighted spanning tree
B. a weighted shortest path from s to t
C. an Euler walk from s to t
D. a Hamiltonian path from s to t

gatecse-2005 algorithms graph-algorithms normal

Answer key

1.17.5 Graph Algorithms: GATE CSE 2016 | Set 2 | Question: 41

In an adjacency list representation of an undirected simple graph $G = (V, E)$, each edge (u, v) has two adjacency list entries: $[v]$ in the adjacency list of u, and $[u]$ in the adjacency list of v. These are called twins of each other. A twin pointer is a pointer from an adjacency list entry to its twin. If $|E| = m$ and $|V| = n$, and the memory size is not a constraint, what is the time complexity of the most efficient algorithm to set the twin pointer in each entry in each adjacency list?

A. $\Theta(n^2)$
B. $\Theta(n+m)$
C. $\Theta(m^2)$
D. $\Theta(n^4)$

gatecse-2016-set2 algorithms graph-algorithms normal

Answer key

1.17.6 Graph Algorithms: GATE CSE 2017 | Set 1 | Question: 26

Let $G = (V, E)$ be *any* connected, undirected, edge-weighted graph. The weights of the edges in E are positive and distinct. Consider the following statements:

I. Minimum Spanning Tree of G is always unique.
II. Shortest path between any two vertices of G is always unique.

Which of the above statements is/are necessarily true?

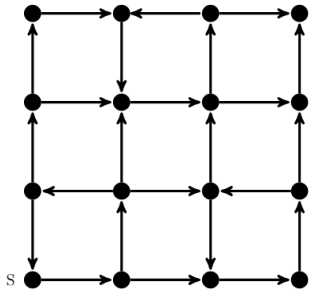
A. I only B. II only C. both I and II D. neither I nor II

gatecse-2017-set1 algorithms graph-algorithms normal

Answer key

1.17.7 Graph Algorithms: GATE CSE 2021 | Set 2 | Question: 46

Consider the following directed graph:



Which of the following is/are correct about the graph?

A. The graph does not have a topological order
B. A depth-first traversal starting at vertex S classifies three directed edges as back edges
C. The graph does not have a strongly connected component
D. For each pair of vertices u and v, there is a directed path from u to v

gatecse-2021-set2 multiple-selects algorithms graph-algorithms two-marks

Answer key

1.17.8 Graph Algorithms: GATE CSE 2026 | Set 1 | Question: 31

Let $G(V, E)$ be an undirected, edge-weighted graph with integer weights. The weight of a path is the sum of the weights of the edges in that path. The length of a path is the number of edges in that path.

Let $s \in V$ be a vertex in G. For every $u \in V$ and for every $k \geq 0$, let $d_k(u)$ denote the weight of a shortest path (in terms of weight) from s to u of length at most k. If there is no path from s to u of length at most k, then $d_k(u) = \infty$.

Consider the statements:

S1: For every $k \geq 0$ and $u \in V, d_{k+1}(u) \leq d_k(u)$.

S2: For every $(u, v) \in E$, if (u, v) is part of a shortest path (in terms of weight) from s to v, then for every $k \geq 0, d_k(u) \leq d_k(v)$.

Which one of the following options is correct?

A. Only $\mathrm{S1}$ is true
B. Only $\mathrm{S2}$ is true
C. Both $\mathrm{S1}$ and $\mathrm{S2}$ are true
D. Neither $\mathrm{S1}$ nor $\mathrm{S2}$ is true

gatecse-2026-set1 algorithms two-marks graph-algorithms

Answer key

1.17.9 Graph Algorithms: GATE IT 2005 | Question: 15

In the following table, the left column contains the names of standard graph algorithms and the right column contains the time complexities of the algorithms. Match each algorithm with its time complexity.

1.	Bellman-Ford algorithm	A:	$O(m \log n)$
2.	Kruskal's algorithm	B:	$O(n^3)$
3.	Floyd-Warshall algorithm	C:	$O(nm)$
4.	Topological sorting	D:	$O(n+m)$

A. $1 \rightarrow \text{C}, 2 \rightarrow \text{A}, 3 \rightarrow \text{B}, 4 \rightarrow \text{D}$
B. $1 \rightarrow \text{B}, 2 \rightarrow \text{D}, 3 \rightarrow \text{C}, 4 \rightarrow \text{A}$
C. $1 \rightarrow \text{C}, 2 \rightarrow \text{D}, 3 \rightarrow \text{A}, 4 \rightarrow \text{B}$
D. $1 \rightarrow \text{B}, 2 \rightarrow \text{A}, 3 \rightarrow \text{C}, 4 \rightarrow \text{D}$

gateit-2005 algorithms graph-algorithms match-the-following easy

Answer key

1.17.10 Graph Algorithms: GATE IT 2005 | Question: 84a

A sink in a directed graph is a vertex i such that there is an edge from every vertex $j \neq i$ to i and there is no edge from i to any other vertex. A directed graph G with n vertices is represented by its adjacency matrix A, where $A[i][j] = 1$ if there is an edge directed from vertex i to j and 0 otherwise. The following algorithm determines whether there is a sink in the graph G.

```
i = 0;
do {
   j = i + 1;
   while ((j < n) && E1) j++;
   if (j < n) E2;
} while (j < n);
flag = 1;
for (j = 0; j < n; j++)
   if ((j! = i) && E3) flag = 0;
if (flag) printf("Sink exists");
else printf ("Sink does not exist");
```

Choose the correct expressions for E_1 and E_2

A. $E_1 : A[i][j]$ and $E_2 : i = j;$
B. $E_1 : \ !A[i][j]$ and $E_2 : i = j + 1;$
C. $E_1 : \ !A[i][j]$ and $E_2 : i = j;$
D. $E_1 : A[i][j]$ and $E_2 : i = j + 1;$

gateit-2005 algorithms graph-algorithms normal

Answer key

1.17.11 Graph Algorithms: GATE IT 2005 | Question: 84b

A sink in a directed graph is a vertex i such that there is an edge from every vertex $j \neq i$ to i and there is no edge from i to any other vertex. A directed graph G with n vertices is represented by its adjacency matrix A, where $A[i][j] = 1$ if there is an edge directed from vertex i to j and 0 otherwise. The following algorithm determines whether there is a sink in the graph G.

```
i = 0;
do {
   j = i + 1;
   while ((j < n) && E1) j++;
   if (j < n) E2;
} while (j < n);
flag = 1;
for (j = 0; j < n; j++)
   if ((j! = i) && E3) flag = 0;
if (flag) printf("Sink exists") ;
else printf ("Sink does not exist");
```

Choose the correct expression for E_3

A. $(A[i][j] \ \&\& \ !A[j][i])$
B. $(!A[i][j] \ \&\& \ A[j][i])$
C. $(!A[i][j] \ || \ A[j][i])$
D. $(A[i][j] \ || \ !A[j][i])$

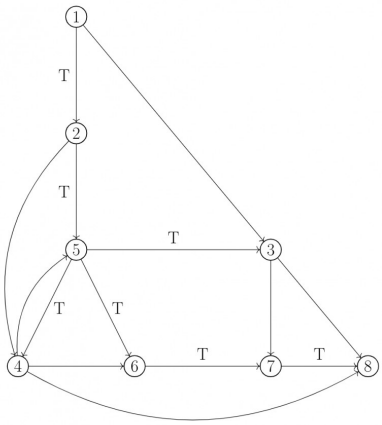
gateit-2005 algorithms graph-algorithms normal

Answer key

1.18 Graph Search (23)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (7Q)

1.18.1 Graph Search: GATE CSE 1989 | Question: 4-vii



In the graph shown above, the depth-first spanning tree edges are marked with a $'T'$. Identify the forward, backward, and cross edges.

gate1989 descriptive algorithms graph-algorithms depth-first-search graph-search

Answer key

1.18.2 Graph Search: GATE CSE 2000 | Question: 1.13

The most appropriate matching for the following pairs

X: depth first search	1: heap
Y: breadth first search	2: queue
Z: sorting	3: stack

is:

A. $\mathrm{X}-1,\ \mathrm{Y}-2,\ \mathrm{Z}-3$
B. $\mathrm{X}-3,\ \mathrm{Y}-1,\ \mathrm{Z}-2$
C. $\mathrm{X}-3,\ \mathrm{Y}-2,\ \mathrm{Z}-1$
D. $\mathrm{X}-2,\ \mathrm{Y}-3,\ \mathrm{Z}-1$

gatecse-2000 algorithms easy graph-algorithms graph-search match-the-following

Answer key

1.18.3 Graph Search: GATE CSE 2000 | Question: 2.19

Let G be an undirected graph. Consider a depth-first traversal of G, and let T be the resulting depth-first search tree. Let u be a vertex in G and let v be the first new (unvisited) vertex visited after visiting u in the traversal. Which of the following statement is always true?

A. $\{u, v\}$ must be an edge in G, and u is a descendant of v in T
B. $\{u, v\}$ must be an edge in G, and v is a descendant of u in T
C. If $\{u, v\}$ is not an edge in G then u is a leaf in T
D. If $\{u, v\}$ is not an edge in G then u and v must have the same parent in T

gatecse-2000 algorithms graph-algorithms normal graph-search

Answer key

1.18.4 Graph Search: GATE CSE 2001 | Question: 2.14

Consider an undirected, unweighted graph G. Let a breadth-first traversal of G be done starting from a node r. Let $d(r, u)$ and $d(r, v)$ be the lengths of the shortest paths from r to u and v respectively in G. If u is visited before v during the breadth-first traversal, which of the following statements is correct?

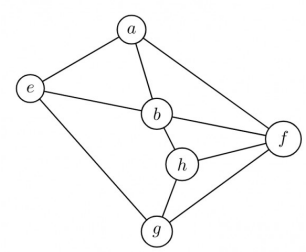
A. $d(r,u) < d(r,v)$
B. $d(r,u) > d(r,v)$
C. $d(r,u) \leq d(r,v)$
D. None of the above

gatecse-2001 algorithms graph-algorithms normal graph-search

Answer key

1.18.5 Graph Search: GATE CSE 2003 | Question: 21

Consider the following graph:



Among the following sequences:

I. abeghf
II. abfehg
III. abfhge
IV. afghbe

Which are the depth-first traversals of the above graph?

A. I, II and IV only
B. I and IV only
C. II, III and IV only
D. I, III and IV only

gatecse-2003 algorithms graph-algorithms normal graph-search

Answer key

1.18.6 Graph Search: GATE CSE 2006 | Question: 48

Let T be a depth first search tree in an undirected graph G. Vertices u and v are leaves of this tree T. The degrees of both u and v in G are at least 2. which one of the following statements is true?

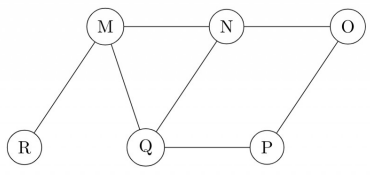
A. There must exist a vertex w adjacent to both u and v in G
B. There must exist a vertex w whose removal disconnects u and v in G
C. There must exist a cycle in G containing u and v
D. There must exist a cycle in G containing u and all its neighbours in G

gatecse-2006 algorithms graph-algorithms normal graph-search

Answer key

1.18.7 Graph Search: GATE CSE 2008 | Question: 19

The Breadth First Search algorithm has been implemented using the queue data structure. One possible order of visiting the nodes of the following graph is:



A. MNOPQR
B. NQMPOR
C. QMNPRO
D. QMNPOR

gatecse-2008 normal algorithms graph-algorithms graph-search

Answer key

1.18.8 Graph Search: GATE CSE 2014 | Set 1 | Question: 11

Let G be a graph with n vertices and m edges. What is the tightest upper bound on the running time of Depth First Search on G, when G is represented as an adjacency matrix?

A. $\Theta(n)$ B. $\Theta(n+m)$ C. $\Theta(n^2)$ D. $\Theta(m^2)$

gatecse-2014-set1 algorithms graph-algorithms normal graph-search

Answer key

1.18.9 Graph Search: GATE CSE 2014 | Set 2 | Question: 14

Consider the tree arcs of a BFS traversal from a source node W in an unweighted, connected, undirected graph. The tree T formed by the tree arcs is a data structure for computing

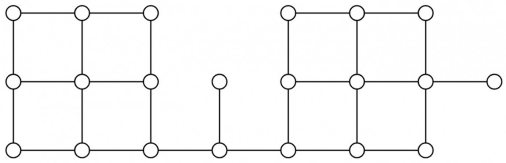
A. the shortest path between every pair of vertices.
B. the shortest path from W to every vertex in the graph.
C. the shortest paths from W to only those nodes that are leaves of T.
D. the longest path in the graph.

gatecse-2014-set2 algorithms graph-algorithms normal graph-search

Answer key

1.18.10 Graph Search: GATE CSE 2014 | Set 3 | Question: 13

Suppose depth first search is executed on the graph below starting at some unknown vertex. Assume that a recursive call to visit a vertex is made only after first checking that the vertex has not been visited earlier. Then the maximum possible recursion depth (including the initial call) is _____.



gatecse-2014-set3 algorithms graph-algorithms numerical-answers normal graph-search

Answer key

1.18.11 Graph Search: GATE CSE 2015 | Set 1 | Question: 45

Let $G = (V, E)$ be a simple undirected graph, and s be a particular vertex in it called the source. For $x \in V$, let $d(x)$ denote the shortest distance in G from s to x. A breadth first search (BFS) is performed starting at s. Let T be the resultant BFS tree. If (u, v) is an edge of G that is not in T, then which one of the following CANNOT be the value of $d(u) - d(v)$?

A. -1 B. 0 C. 1 D. 2

gatecse-2015-set1 algorithms graph-algorithms normal graph-search

Answer key

1.18.12 Graph Search: GATE CSE 2016 | Set 2 | Question: 11

Breadth First Search (BFS) is started on a binary tree beginning from the root vertex. There is a vertex t at a distance four from the root. If t is the n^{th} vertex in this BFS traversal, then the maximum possible value of n is _____

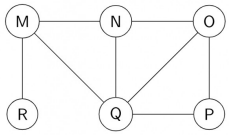
gatecse-2016-set2 algorithms graph-algorithms normal numerical-answers graph-search

Answer key

1.18.13 Graph Search: GATE CSE 2017 | Set 2 | Question: 15

The Breadth First Search (BFS) algorithm has been implemented using the queue data structure. Which one

of the following is a possible order of visiting the nodes in the graph below?



A. MNOPQR B. NQMPOR C. QMNROP D. POQNMR

gatecse-2017-set2 algorithms graph-algorithms graph-search

Answer key

1.18.14 Graph Search: GATE CSE 2018 | Question: 30

Let G be a simple undirected graph. Let T_D be a depth first search tree of G. Let T_B be a breadth first search tree of G. Consider the following statements.

I. No edge of G is a cross edge with respect to T_D. (A cross edge in G is between two nodes neither of which is an ancestor of the other in T_D).
II. For every edge (u, v) of G, if u is at depth i and v is at depth j in T_B, then $|i - j| = 1$.

Which of the statements above must necessarily be true?

A. I only B. II only C. Both I and II D. Neither I nor II

gatecse-2018 algorithms graph-algorithms graph-search normal two-marks

Answer key

1.18.15 Graph Search: GATE CSE 2023 | Question: 46

Let $U = \{1, 2, 3\}$. Let 2^U denote the powerset of U. Consider an undirected graph G whose vertex set is 2^U. For any $A, B \in 2^U$, (A, B) is an edge in G if and only if (i) $A \neq B$, and (ii) either $A \subsetneq B$ or $B \subsetneq A$. For any vertex A in G, the set of all possible orderings in which the vertices of G can be visited in a Breadth First Search (BFS) starting from A is denoted by $\mathcal{B}(A)$.

If $\emptyset$ denotes the empty set, then the cardinality of $\mathcal{B}(\emptyset)$ is _____.

gatecse-2023 algorithms breadth-first-search numerical-answers two-marks graph-search

Answer key

1.18.16 Graph Search: GATE CSE 2024 | Set 1 | Question: 35

Let G be a directed graph and T a depth first search (DFS) spanning tree in G that is rooted at a vertex v. Suppose T is also a breadth first search (BFS) tree in G, rooted at v. Which of the following statements is/are TRUE for *every* such graph G and tree T?

A. There are no back-edges in G with respect to the tree T
B. There are no cross-edges in G with respect to the tree T
C. There are no forward-edge in G with respect to the tree T
D. The only edges in G are the edges in T

gatecse-2024-set1 algorithms multiple-selects graph-search two-marks

Answer key

1.18.17 Graph Search: GATE CSE 2024 | Set 1 | Question: 50

The number of edges present in the forest generated by the DFS traversal of an undirected graph G with 100 vertices is 40. The number of connected components in G is _____.

gatecse-2024-set1 numerical-answers graph-search graph-algorithms two-marks

Answer key

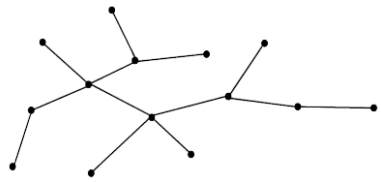
1.18.18 Graph Search: GATE CSE 2025 | Set 2 | Question: 49

Consider the following algorithm **someAlgo** that takes an undirected graph G as input.

someAlgo (G)

1. Let v be any vertex in G. Run BFS on G starting at v. Let u be a vertex in G at maximum distance from v as given by the BFS.
2. Run BFS on G again with u as the starting vertex. Let z be the vertex at maximum distance from u as given by the BFS.
3. Output the distance between u and z in G.

The output of **someAlgo** (T) for the tree shown in the given figure is _____. (Answer in integer)



gatecse2025-set2 algorithms breadth-first-search graph-search numerical-answers two-marks

Answer key

1.18.19 Graph Search: GATE DS&AI 2024 | Question: 4

Consider performing depth-first search (DFS) on an undirected and unweighted graph G starting at vertex s. For any vertex u in $G, d[u]$ is the length of the shortest path from s to u. Let (u, v) be an edge in G such that $d[u] < d[v]$. If the edge (u, v) is explored first in the direction from u to v during the above DFS, then (u, v) becomes a _____ edge.

A. tree B. cross C. back D. gray

gate-ds-ai-2024 graph-search depth-first-search algorithms one-mark

Answer key

1.18.20 Graph Search: GATE IT 2005 | Question: 14

In a depth-first traversal of a graph G with n vertices, k edges are marked as tree edges. The number of connected components in G is

A. k B. $k+1$ C. $n-k-1$ D. $n-k$

gateit-2005 algorithms graph-algorithms normal graph-search

Answer key

1.18.21 Graph Search: GATE IT 2006 | Question: 47

Consider the depth-first-search of an undirected graph with 3 vertices P, Q, and R. Let discovery time $d(u)$ represent the time instant when the vertex u is first visited, and finish time $f(u)$ represent the time instant when the vertex u is last visited. Given that

$d(P) = 5$ units	$f(P) = 12$ units
$d(Q) = 6$ units	$f(Q) = 10$ units
$d(R) = 14$ unit	$f(R) = 18$ units

Which one of the following statements is TRUE about the graph?

A. There is only one connected component
B. There are two connected components, and P and R are connected
C. There are two connected components, and Q and R are connected
D. There are two connected components, and P and Q are connected